

INTELLIGENCE ARTIFICIELLE

2

APPRENTISSAGE AUTOMATIQUE

Dr Pascal FAYE

Email: pascal.faye@ussein.edu.sn

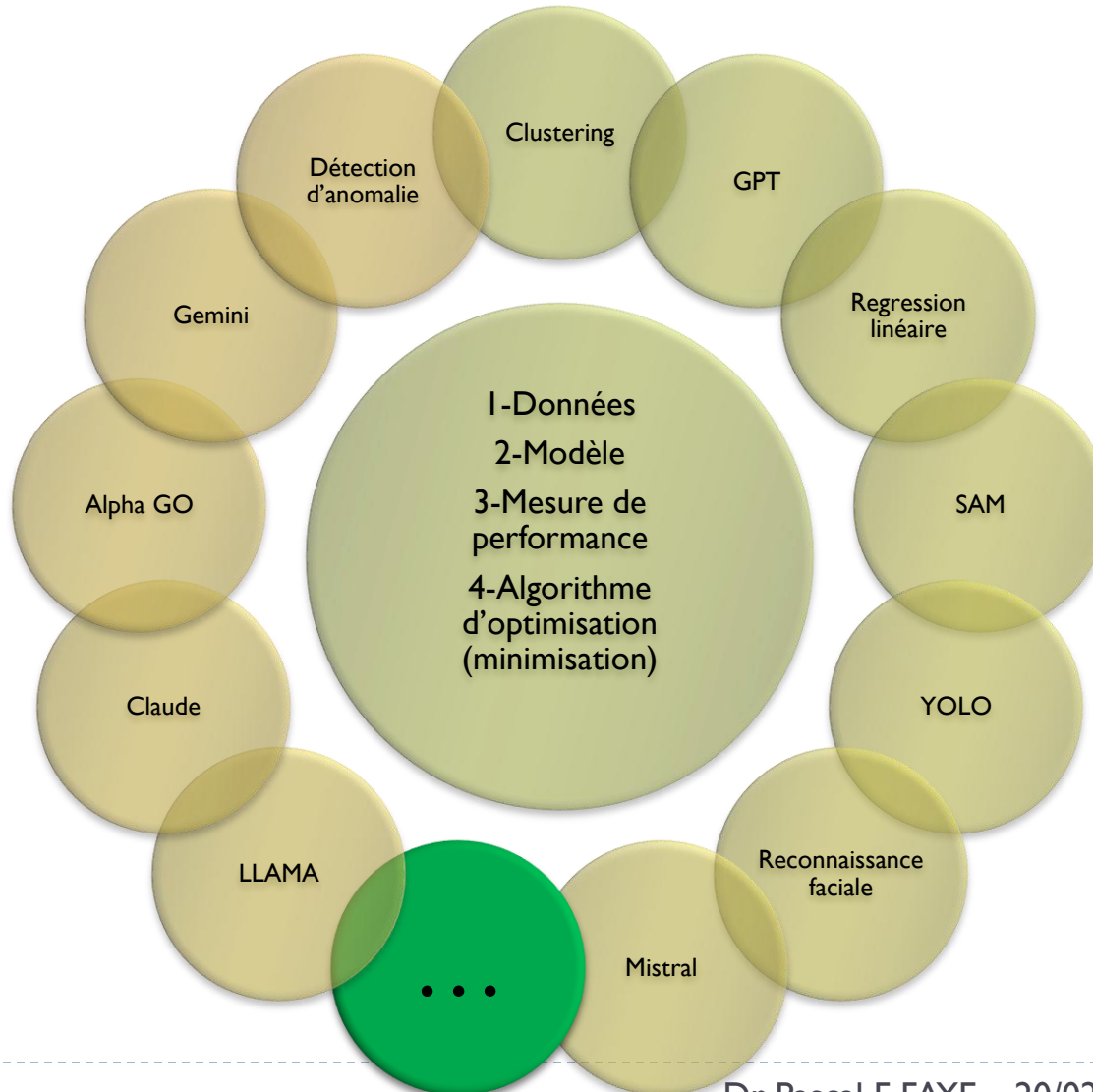
Plan

- ▶ Introduction
- ▶ Méthodes (types) d'apprentissage
 - Supervisé
 - Non-supervisé
 - Par renforcement

Introduction

- ▶ L'apprentissage automatique fait référence au développement, à l'analyse et à l'implémentation de méthodes qui permettent à une **entité!!!** d'évoluer et ainsi de remplir des tâches qu'il est difficile ou impossible de remplir par des algorithmiques classiques.
- ▶ L'apprentissage automatique en IA, s'intéresse donc, à la façon dont l'agent peut arriver à acquérir ses connaissances cohérentes à partir de son expérience ou de son environnement.
- ▶ On dira qu'un agent apprend dès lors qu'elle change sa structure, son programme ou ses données en fonction de données en entrée ou de son environnement de sorte que ses performances futures deviennent meilleures.
- ▶ Exemple:
 - Un robot ayant la capacité de bouger ses membres mais ne sachant initialement rien de la coordination des mouvements permettant la marche.
 - Un robot peut apprendre à reconnaître des caractères en observant des exemples.
 - Etc.

Notion de base des algorithmes IA

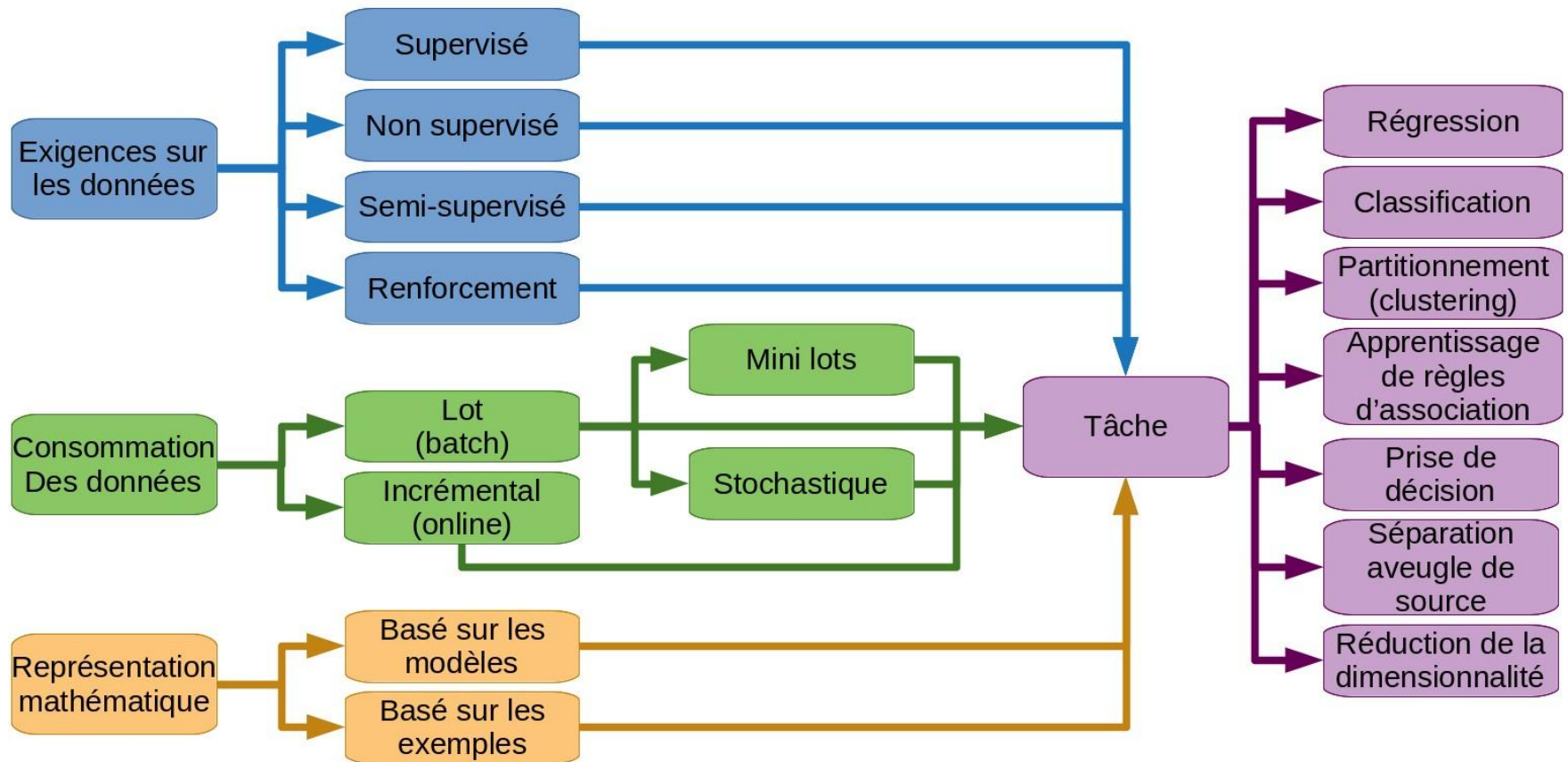


Méthodes (types) d'apprentissages

- ▶ Tout apprentissage s'opère à partir d'exemples de données fournies par un expert ou non.
- ▶ Nous pouvons distinguer trois modes d'apprentissage complémentaires selon les types d'informations dont dispose l'agent:



Méthodes (types) d'apprentissages



TAXONOMIE DE L'APPRENTISSAGE AUTOMATIQUE

1. <https://hermit-notebook.site/fr/notebook/computer-sciences/artificial-intelligence/machine-learning/taxonomy-of-machine-learning/>
2. <https://openclassrooms.com/fr/courses/4011851-initiez-vous-au-machine-learning/4020611-identifiez-les-differents-types-dapprentissage-automatiques>

Apprentissage supervisé

- ▶ L'agent doit apprendre à classer des instances à partir d'un ensemble d'exemples qui sont étiquetés par un expert humain.
- ▶ Un exemple typique de l'apprentissage supervisé serait donc la reconnaissance visuelle des chiffres :
 - ▶ on présente à l'agent un ensemble de chiffres manuscrits (étiquette par les nombres qu'ils représentent) et il doit apprendre à identifier les différents chiffres.
- ▶ Les principaux algorithmes sont les **réseaux de neurones**, les **machines à vecteur de support (SVM)**, et les **arbres de décision**.

Apprentissage supervisé: les arbres de décision

- ▶ Cette méthode consiste à construire un arbre de décision à partir d'un ensemble d'exemples.
- ▶ Cet arbre est ensuite utilisé pour classer de nouvelles instances.
- ▶ L'avantage principal de cette approche est sa simplicité de construction et d'interprétation.
- ▶ Formellement
- ▶ $f : A_i \times A_j \times \dots \times A_n \rightarrow C$
 - ▶ Où f est la fonction de classement des données par rapport à la classe C et des exemples appartenant à l'ensemble de attributs $A_i \times A_j \times \dots \times A_n$.
- ▶ Chaque attribut A_i a un domaine de valeurs possibles.

Apprentissage supervisé: les arbres de décision

- ▶ L'algorithme que nous présentons ici est l'algorithme classique
De nombreuses extensions ou modifications existent.

- ▶ D'une part,
 - il y a des extensions de cette approche aux problèmes ayant des attributs dont les domaines sont numériques (e.g. Salaire, Poids, Température).
 - Il y a également des extensions pour prendre en compte des exemples dont les valeurs de certains des attributs sont inconnus .
- ▶ D'autre part, il y a des travaux visant à améliorer la qualité des arbres produits.
 - Une possibilité est de ne pas continuer à construire une branche si aucun attribut ne donne un gain d'information significatif.
 - Une autre méthode consiste à ajouter une dernière étape à l'algorithme où certaines parties de l'arbre sont supprimées, afin de trouver un arbre plus simple.
 - Une troisième possibilité est de produire plusieurs arbres de décision différents et de les utiliser tous ensemble pour classer les instances.

Apprentissage supervisé: les arbres de décision

Exemple

- Nous cherchons à détecter si **un mail est un spam ou non**. Nous disposons des données suivantes:

Nous avons les quatre attributs

Auteur (si l'auteur du mail est connu ou non),

MotClés (si le message contient un ensemble de mots associé aux spams),

HTML (si le mail contient un lien html),

Majuscule (si le message contient des mots en majuscules ou non)

Spam (qui est l'attribut cible).

	Auteur	MotClés	HTML	Majuscule	Spam
1	<i>connu</i>	<i>oui</i>	<i>oui</i>	<i>non</i>	<i>non</i>
2	<i>inconnu</i>	<i>non</i>	<i>non</i>	<i>oui</i>	<i>oui</i>
3	<i>inconnu</i>	<i>oui</i>	<i>non</i>	<i>non</i>	<i>non</i>
4	<i>inconnu</i>	<i>oui</i>	<i>oui</i>	<i>oui</i>	<i>oui</i>
5	<i>connu</i>	<i>non</i>	<i>non</i>	<i>non</i>	<i>non</i>
6	<i>inconnu</i>	<i>non</i>	<i>oui</i>	<i>oui</i>	<i>oui</i>
7	<i>connu</i>	<i>non</i>	<i>non</i>	<i>oui</i>	<i>non</i>
8	<i>inconnu</i>	<i>oui</i>	<i>oui</i>	<i>non</i>	<i>oui</i>
9	<i>connu</i>	<i>oui</i>	<i>non</i>	<i>non</i>	<i>non</i>
10	<i>inconnu</i>	<i>oui</i>	<i>non</i>	<i>oui</i>	<i>oui</i>
11	<i>connu</i>	<i>oui</i>	<i>non</i>	<i>non</i>	<i>non</i>
12	<i>inconnu</i>	<i>non</i>	<i>non</i>	<i>non</i>	<i>non</i>

Les domaines des attributs sont :

dom(Auteur) = {connu, inconnu}, **dom(MotClés)** = {oui, non}, **dom(HTML)** = {oui, non}, **dom(Majuscule)** = {oui, non}, et **dom(Spam)** = {oui, non}.

Apprentissage supervisé: les arbres de décision

Exemple

Principe de création de l'arbre de décision

- ▶ Les nœuds non-terminaux sont étiquetés par des attributs (non-cibles),
- ▶ Les arcs sont étiquetés par une valeur dans le domaine de l'attribut associé au nœud parent,
- ▶ Chaque feuille de l'arbre prend une valeur dans le domaine de l'attribut cible.
- ▶ Un arbre de décision définit la valeur de l'attribut cible pour chaque instance possible.
- ▶ Pour trouver la valeur d'une instance spécifique, il suffit de parcourir l'arbre en choisissant l'arc qui correspond à l'instance en question jusqu'à arriver à une feuille dont l'étiquette donne la classe de l'instance.

Apprentissage supervisé: les arbres de décision

Exemple

Principe de création de l'arbre de décision

NB: À chaque ensemble d'exemples correspond plusieurs arbres de décisions.

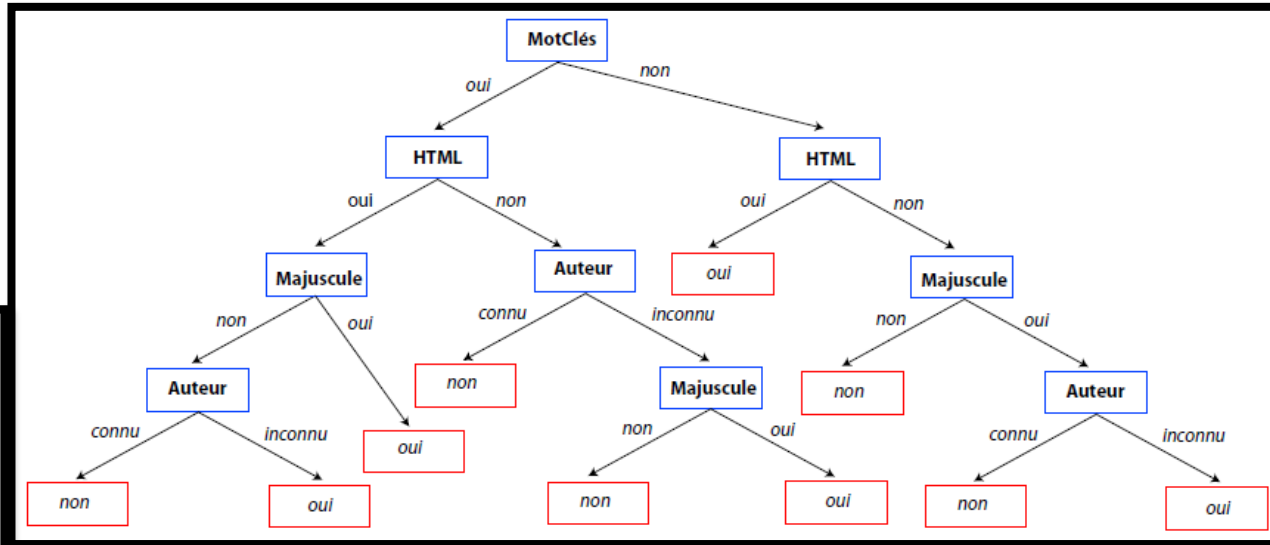
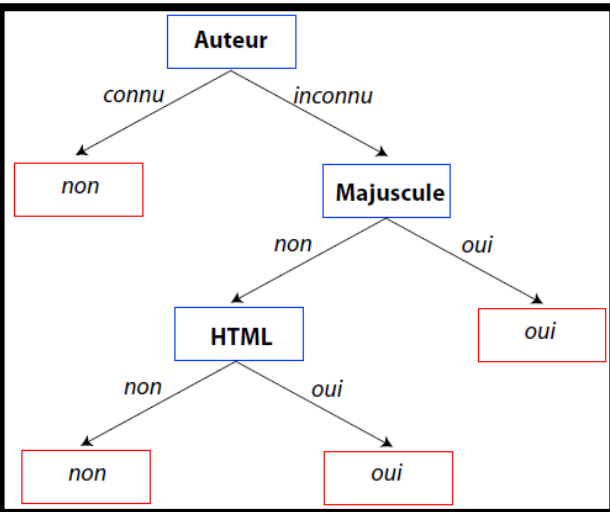
- ▶ Donc, comment savoir si un arbre de décision est un bon choix pour un ensemble de données ?
- ▶ Un premier critère est **le taux de couverture de l'arbre**, i.e. le pourcentage des exemples qui sont bien classés par l'arbre.
 - ▶ L'importance de ce critère devrait être évidente, car les exemples dont nous disposons sont nos seules informations sur la fonction à apprendre.
- ▶ Mais aussi la simplicité de l'arbre (qui peut être mesurée par le nombre de nœuds non-terminaux) pour éviter aussi le problème de surapprentissage.

Apprentissage supervisé: les arbres de décision

Exemple

Principe de création de l'arbre de décision

► Comparaison???



Lequel???
ou autre???

Apprentissage supervisé: les arbres de décision

Exemple

Principe de création de l'arbre de décision

- ▶ Alors, comment construire des arbres de décision?
 - ▶ Utilisation de la notion de gain d'information basé sur la notion d'entropie, qui est probablement l'heuristique la plus connue pour les arbres de décision.
 - ▶ L'entropie est une quantité qui mesure l'information (et donc nous sera utile pour évaluer les gains d'information), c'est-à-dire, comment les différentes classes sont présentées.
 - ▶ L'entropie $H(D)$ d'un ensemble D est minimale quand tous les exemples contenus dans D appartiennent à la même classe. Elle est maximale quand les exemples de D sont uniformément répartis parmi les différentes classes.

Apprentissage supervisé: les arbres de décision

Exemple

Principe de création de l'arbre de décision

► Alors, comment construire des arbres de décision?

- Si nous avons n classes différentes, l'entropie $H(D)$ d'un ensemble D est définie par :

$$H(D) = - (p_1 \log_2 p_1 + p_2 \log_2 p_2 + \dots + p_n \log_2 p_n),$$

où p_i donne la proportion d'éléments dans D qui sont étiquetés par la i -ème classe.

- Le gain d'information $G(A,D)$ d'un attribut par rapport à un autre attribut A représente la différence entre l'entropie avant et après le branchement sur A . Plus le gain est important, plus l'attribut est utile pour séparer les exemples en classes.

$$G(A, D) = H(D) - \left(\sum_{x \in \text{dom}(A)} \frac{|D[A = x]|}{|D|} H(D[A = x]) \right)$$

où, $D[A = x]$ signifie les éléments de D ayant la valeur x pour l'attribut A .

Apprentissage supervisé: les arbres de décision

Exemple

Principe de création de l'arbre de décision

- ▶ **Illustration** : considérons $A = \{\mathbf{Auteur}, \mathbf{MotClés}, \mathbf{HTML}, \mathbf{Majuscule}\}$, $C = \text{Spam}$, et D les douze exemples ci-dessous.
- ▶ Comme les exemples ne sont pas tous dans la même classe, et A n'est pas vide, nous devons calculer l'attribut avec le plus grand gain d'information.
- ▶ Tout d'abord il nous faut trouver l'entropie de D :

$$H(D) = -\left(\frac{5}{12} \log_2 \frac{5}{12} + \frac{7}{12} \log_2 \frac{7}{12}\right) = 0,98$$

	Auteur	MotClés	HTML	Majuscule	Spam
1	<i>connu</i>	<i>oui</i>	<i>oui</i>	<i>non</i>	<i>non</i>
2	<i>inconnu</i>	<i>non</i>	<i>non</i>	<i>oui</i>	<i>oui</i>
3	<i>inconnu</i>	<i>oui</i>	<i>non</i>	<i>non</i>	<i>non</i>
4	<i>inconnu</i>	<i>oui</i>	<i>oui</i>	<i>oui</i>	<i>oui</i>
5	<i>connu</i>	<i>non</i>	<i>non</i>	<i>non</i>	<i>non</i>
6	<i>inconnu</i>	<i>non</i>	<i>oui</i>	<i>oui</i>	<i>oui</i>
7	<i>connu</i>	<i>non</i>	<i>non</i>	<i>oui</i>	<i>non</i>
8	<i>inconnu</i>	<i>oui</i>	<i>oui</i>	<i>non</i>	<i>oui</i>
9	<i>connu</i>	<i>oui</i>	<i>non</i>	<i>non</i>	<i>non</i>
10	<i>inconnu</i>	<i>oui</i>	<i>non</i>	<i>oui</i>	<i>oui</i>
11	<i>connu</i>	<i>oui</i>	<i>non</i>	<i>non</i>	<i>non</i>
12	<i>inconnu</i>	<i>non</i>	<i>non</i>	<i>non</i>	<i>non</i>

Apprentissage supervisé: les arbres de décision

Exemple

Principe de création de l'arbre de décision

► **Illustration :**

- Les entropies pour les différentes valeurs de **Auteur**. Pour la valeur connu, tous les exemples ont l'étiquette non, donc nous obtiendrons une entropie de 0 :

$$H(D[\text{Auteur} = \text{connu}]) = -\left(\frac{0}{5} \log_2 \frac{0}{5} + \frac{5}{5} \log_2 \frac{5}{5}\right) = 0$$

- Pour inconnu, il y a 5 exemples oui et 2 exemples non, ce qui nous donne une valeur de 0,86

$$H(D[\text{Auteur} = \text{inconnu}]) = -\left(\frac{5}{7} \log_2 \frac{5}{7} + \frac{2}{7} \log_2 \frac{2}{7}\right) = 0,86$$

- Nous trouvons un gain d'information de 0,48 pour l'attribut Auteur :

$$G(\text{Auteur}, D) = 0,98 - \left(\frac{5}{12} * 0 + \frac{7}{12} * 0,86\right) = 0,48$$

Apprentissage supervisé: les arbres de décision

Exemple

Principe de création de l'arbre de décision

► Illustration

- Cherchons maintenant le gain d'informations de l'attribut **MotClés** :

$$H(D[\text{MotClés} = \text{oui}]) = -\left(\frac{3}{7} \log_2 \frac{3}{7} + \frac{4}{7} \log_2 \frac{4}{7}\right) = 0,99$$

$$H(D[\text{MotClés} = \text{non}]) = -\left(\frac{2}{5} \log_2 \frac{2}{5} + \frac{3}{5} \log_2 \frac{3}{5}\right) = 0,97$$

$$G(\text{MotClés}, D) = 0,98 - \left(\frac{7}{12} * 0,99 + \frac{5}{12} * 0,97\right) = 0$$

	Auteur	MotClés	HTML	Majuscule	Spam
1	connu	oui	oui	non	non
2	inconnu	non	non	oui	oui
3	inconnu	oui	non	non	non
4	inconnu	oui	oui	oui	oui
5	connu	non	non	non	non
6	inconnu	non	oui	oui	oui
7	connu	non	non	oui	non
8	inconnu	oui	oui	non	oui
9	connu	oui	non	non	non
10	inconnu	oui	non	oui	oui
11	connu	oui	non	non	non
12	inconnu	non	non	non	non

Apprentissage supervisé: les arbres de décision

Exemple

Principe de création de l'arbre de décision

► Illustration

- Cherchons maintenant le gain d'informations de l'attribut **HTML** :

$$H(D[\text{HTML} = \text{oui}]) = -\left(\frac{3}{4} \log_2 \frac{3}{4} + \frac{1}{4} \log_2 \frac{1}{4}\right) = 0,81$$

$$H(D[\text{HTML} = \text{non}]) = -\left(\frac{2}{8} \log_2 \frac{2}{8} + \frac{6}{8} \log_2 \frac{6}{8}\right) = 0,81$$

$$G(\text{HTML}, D) = 0,98 - \left(\frac{4}{12} * 0,81 + \frac{8}{12} * 0,81\right) = 0,17$$

	Auteur	MotClés	HTML	Majuscule	Spam
1	<i>connu</i>	<i>oui</i>	<i>oui</i>	<i>non</i>	<i>non</i>
2	<i>inconnu</i>	<i>non</i>	<i>non</i>	<i>oui</i>	<i>oui</i>
3	<i>inconnu</i>	<i>oui</i>	<i>non</i>	<i>non</i>	<i>non</i>
4	<i>inconnu</i>	<i>oui</i>	<i>oui</i>	<i>oui</i>	<i>oui</i>
5	<i>connu</i>	<i>non</i>	<i>non</i>	<i>non</i>	<i>non</i>
6	<i>inconnu</i>	<i>non</i>	<i>oui</i>	<i>oui</i>	<i>oui</i>
7	<i>connu</i>	<i>non</i>	<i>non</i>	<i>oui</i>	<i>non</i>
8	<i>inconnu</i>	<i>oui</i>	<i>oui</i>	<i>non</i>	<i>oui</i>
9	<i>connu</i>	<i>oui</i>	<i>non</i>	<i>non</i>	<i>non</i>
10	<i>inconnu</i>	<i>oui</i>	<i>non</i>	<i>oui</i>	<i>oui</i>
11	<i>connu</i>	<i>oui</i>	<i>non</i>	<i>non</i>	<i>non</i>
12	<i>inconnu</i>	<i>non</i>	<i>non</i>	<i>non</i>	<i>non</i>

Apprentissage supervisé: les arbres de décision

Exemple

Principe de création de l'arbre de décision

► Illustration

- Cherchons maintenant le gain d'informations de l'attribut **Majuscule**:

$$H(D[\text{Majuscule} = \text{oui}]) = -\left(\frac{4}{5} \log_2 \frac{4}{5} + \frac{1}{5} \log_2 \frac{1}{5}\right) = 0,72$$

$$H(D[\text{Majuscule} = \text{non}]) = -\left(\frac{1}{7} \log_2 \frac{1}{7} + \frac{6}{7} \log_2 \frac{6}{7}\right) = 0,59$$

$$G(\text{Majuscule}, D) = 0,98 - \left(\frac{5}{12} * 0,72 + \frac{7}{12} * 0,59\right) = 0,33$$

	Auteur	MotClés	HTML	Majuscule	Spam
1	connu	oui	oui	non	non
2	inconnu	non	non	oui	oui
3	inconnu	oui	non	non	non
4	inconnu	oui	oui	oui	oui
5	connu	non	non	non	non
6	inconnu	non	oui	oui	oui
7	connu	non	non	oui	non
8	inconnu	oui	oui	non	oui
9	connu	oui	non	non	non
10	inconnu	oui	non	oui	oui
11	connu	oui	non	non	non
12	inconnu	non	non	non	non

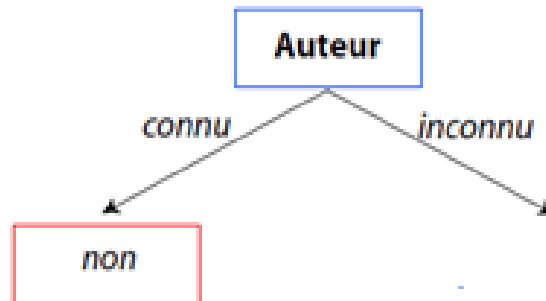
Apprentissage supervisé: les arbres de décision

Exemple

Principe de création de l'arbre de décision

► Illustration

- C'est donc l'attribut **Auteur** qui a le plus grand gain d'information, donc nous utilisons cet attribut pour commencer notre arbre.
- Comme tous les exemples ayant **Auteur=connu** ont tous la même valeur (=non), nous ajoutons un arc marqué connu et un nœud étiqueté par non.
- Pour **Auteur=inconnu**, nous devons appliquer à nouveau l'algorithme pour choisir un attribut parmi les attributs restant pour cette branche de l'arbre.



Apprentissage supervisé: les arbres de décision

Exemple

Principe de création de l'arbre de décision

► Illustration

- Nous travaillons maintenant sur les exemples dont l'auteur est inconnu, i.e. nous mettons :

$D = D[\text{Auteur} = \text{inconnu}]$

- Nous avons déjà calculé l'entropie de cet ensemble : $H(D[\text{Auteur} = \text{inconnu}]) = 0,86$.
- Maintenant il faut déterminer les gains d'information respectifs pour les trois attributs restant.

	Auteur	MotClés	HTML	Majuscule	Spam
1	<i>connu</i>	<i>oui</i>	<i>oui</i>	<i>non</i>	<i>non</i>
2	<i>inconnu</i>	<i>non</i>	<i>non</i>	<i>oui</i>	<i>oui</i>
3	<i>inconnu</i>	<i>oui</i>	<i>non</i>	<i>non</i>	<i>non</i>
4	<i>inconnu</i>	<i>oui</i>	<i>oui</i>	<i>oui</i>	<i>oui</i>
5	<i>connu</i>	<i>non</i>	<i>non</i>	<i>non</i>	<i>non</i>
6	<i>inconnu</i>	<i>non</i>	<i>oui</i>	<i>oui</i>	<i>oui</i>
7	<i>connu</i>	<i>non</i>	<i>non</i>	<i>oui</i>	<i>non</i>
8	<i>inconnu</i>	<i>oui</i>	<i>oui</i>	<i>non</i>	<i>oui</i>
9	<i>connu</i>	<i>oui</i>	<i>non</i>	<i>non</i>	<i>non</i>
10	<i>inconnu</i>	<i>oui</i>	<i>non</i>	<i>oui</i>	<i>oui</i>
11	<i>connu</i>	<i>oui</i>	<i>non</i>	<i>non</i>	<i>non</i>
12	<i>inconnu</i>	<i>non</i>	<i>non</i>	<i>non</i>	<i>non</i>

$$H(D[\text{Auteur} = \text{inconnu}]) = -\left(\frac{5}{7} \log_2 \frac{5}{7} + \frac{2}{7} \log_2 \frac{2}{7}\right) = 0,86$$

Apprentissage supervisé: les arbres de décision

Exemple

Principe de création de l'arbre de décision

► Illustration

$$H(D[\text{Auteur} = \text{inconnu}][\text{MotClés} = \text{oui}]) = -\left(\frac{3}{4} \log_2 \frac{3}{4} + \frac{1}{4} \log_2 \frac{1}{4}\right) = 0,81$$

► MotClés

$$H(D[\text{Auteur} = \text{inconnu}][\text{MotClés} = \text{non}]) = -\left(\frac{2}{3} \log_2 \frac{2}{3} + \frac{1}{3} \log_2 \frac{1}{3}\right) = 0,92$$

$$G(\text{MotClés}, D[\text{Auteur} = \text{inconnu}]) = 0,86 - \left(\frac{4}{7} * 0,81 + \frac{3}{7} * 0,92\right) = 0,01$$

► HTML

$$H(D[\text{Auteur} = \text{inconnu}][\text{HTML} = \text{oui}]) = -\left(\frac{3}{3} \log_2 \frac{3}{3} + \frac{0}{3} \log_2 \frac{0}{3}\right) = 0$$

$$H(D[\text{Auteur} = \text{inconnu}][\text{HTML} = \text{non}]) = -\left(\frac{2}{4} \log_2 \frac{2}{4} + \frac{2}{4} \log_2 \frac{2}{4}\right) = 1$$

$$G(\text{HTML}, D[\text{Auteur} = \text{inconnu}]) = 0,86 - \left(\frac{3}{7} * 0 + \frac{4}{7} * 1\right) = 0,29$$

► Majuscule

$$H(D[\text{Auteur} = \text{inconnu}][\text{Majuscule} = \text{oui}]) = -\left(\frac{4}{4} \log_2 \frac{4}{4} + \frac{0}{4} \log_2 \frac{0}{4}\right) = 0$$

$$H(D[\text{Auteur} = \text{inconnu}][\text{Majuscule} = \text{non}]) = -\left(\frac{1}{3} \log_2 \frac{1}{3} + \frac{2}{3} \log_2 \frac{2}{3}\right) = 0,92$$

$$G(\text{Majuscule}, D[\text{Auteur} = \text{inconnu}]) = 0,86 - \left(\frac{4}{7} * 0 + \frac{3}{7} * 0,92\right) = 0,47$$

Apprentissage supervisé: les arbres de décision

Exemple

Principe de création de l'arbre de décision

► Illustration

- C'est donc l'attribut **Majuscule** qui a le meilleur gain d'information.
- Nous étiquetons donc notre noeud par **Majuscule**, et nous ajoutons deux arcs,
 - l'un pour **Majuscule=oui** et
 - l'autre pour **Majuscule=non**
- Les exemples ayant **Majuscule=oui** ont **tous la valeur oui**, donc nous ajoutons un noeud avec l'étiquette **oui**.
- Pour **Majuscule=non**, nous avons toujours des exemples avec des étiquettes différentes, donc nous continuons.

Apprentissage supervisé: les arbres de décision

Exemple

Principe de création de l'arbre de décision

► Illustration

- Maintenant nous ne considérons que les exemples de D ayant $Auteur = inconnu$ et $Majuscule = non$.
- L'entropie de cet ensemble à été déterminée auparavant :
 $H(D[Auteur = inconnu][Majuscule = non]) = 0,92$.
- Il nous reste que deux attributs possibles : $HTML$ et $MotClés$.
- Après calcul nous trouvons:

$$G(HTML, D[Auteur = inconnu][Majuscule = non]) = 0,92$$

$$G(MotClés, D[Auteur = inconnu][Majuscule = non]) = 0,25$$

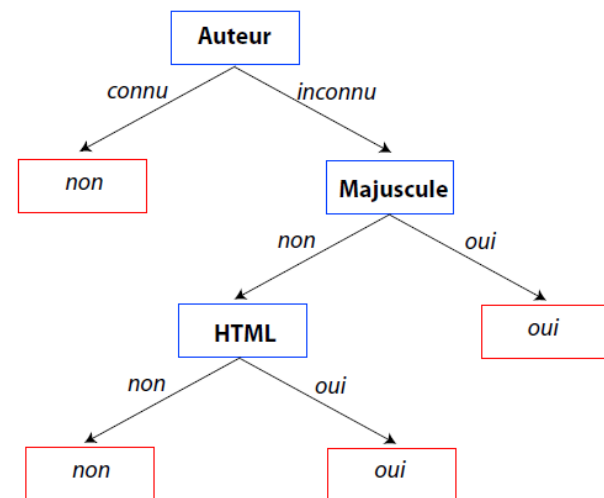
Apprentissage supervisé: les arbres de décision

Exemple

Principe de création de l'arbre de décision

► Illustration

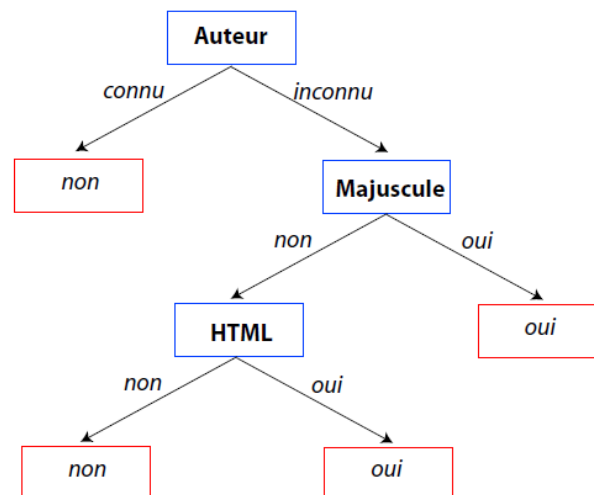
- Nous donnons donc l'étiquette **HTML** à notre noeud, et nous créons deux noeuds fils,
 - l'un pour HTML=oui (qui aura l'étiquette oui)
 - et l'autre pour HTML = non (avec l'étiquette non).
- Ce qui donne l'arbre suivant:



Apprentissage supervisé: les arbres de décision

Exemple : Déroulement sur une instance

- ▶ Prenons l'instance **(inconnu, non, oui, non)**.
- ▶ Nous commençons à la racine qui correspond à l'attribut **Auteur**.
- ▶ Comme notre instance prend la valeur inconnu pour l'attribut **Auteur**, nous suivons l'arc droit.
- ▶ Puis comme le prochain attribut est **Majuscule** et notre instance a la valeur non pour **Majuscule**, nous prenons l'arc gauche.
- ▶ À l'attribut **HTML**, nous choisissons l'arc droit (dont la valeur est oui) et nous trouvons une feuille marquée oui.
- ▶ L'instance (inconnu, non, oui, non) est donc classée comme un mail spam par notre arbre de décision.



Apprentissage supervisé: les arbres de décision

Exemple : algorithme

- ▶ Les arguments de l'algorithme sont un ensemble de attributs $\mathcal{A}$, l'attribut cible C , et un (multi)-ensemble non-vide D d'exemples étiquetés par des valeurs de $\text{dom}(C)$.

```
fonction ARBRE( $\mathcal{A}$ ,  $C$ ,  $D$ )  
  créer un noeud racine  $R$   
  si tous les éléments de  $D$  ont l'étiquette  $c$  alors  
    étiqueter  $R$  par  $c$   
  sinon si  $\mathcal{A}$  est vide  
    étiqueter  $R$  avec la valeur de  $C$  la plus fréquente dans  $D$   
  sinon  
    étiqueter  $R$  par l'attribut  $A \in \mathcal{A}$  avec le plus grand gain d'information  
    pour chaque  $a \in \text{dom}(A)$  faire  
      ajouter un arc avec étiquette  $a$   
      si  $D[A = a] = \emptyset$   
        ajouter à la fin de l'arc un noeud étiqueté par la valeur de  $C$   
          la plus fréquente dans  $D$   
      sinon  
        mettre l'arbre  $\text{ARBRE}(\mathcal{A} \setminus A, C, D[A = a])$  à la fin de la branche  
    retourner l'arbre  $R$ 
```

Code python

```
#Importation des modules nécessaires
from sklearn.tree import DecisionTreeClassifier, plot_tree
import pandas as pd
from sklearn.model_selection import train_test_split
import matplotlib.pyplot as plt
from sklearn import metrics
#importation du dataset
inputData = pd.read_csv("diabetes.csv")
#Déclaration de l'arbre de décision
model = DecisionTreeClassifier(max_depth=2)
#Entraînement de l'arbre de décision
X=inputData[['preg','plas','pres','skin','insu','mass','pedi','age']]
X.values
Y=inputData['class'].values
X_train,X_test,Y_train,
Y_test=train_test_split(X,Y,test_size=0.1,random_state=2)
```

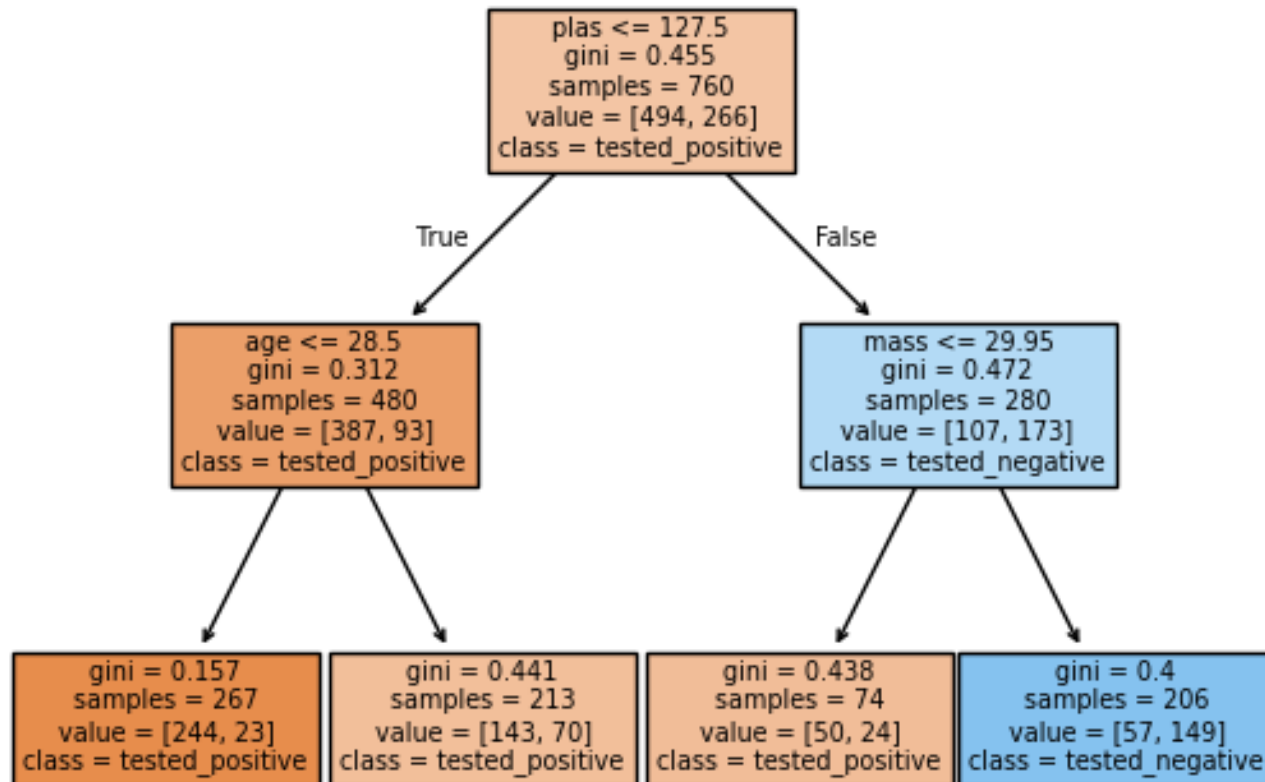
```
print('train set',X_train.shape,Y_train.shape)
print('test set',X_test.shape,Y_test.shape)
model.fit(X_train,Y_train)
print("Précision",model.score(X_test,Y_test))
print("Evaluation test",model.predict(X_test))
#Afficher la matrice de confusion
#confusion_matrix =
metrics.confusion_matrix(Y_test,model.predict(X_test))
#cm_display =
metrics.ConfusionMatrixDisplay(confusion_matrix =
confusion_matrix, display_labels = [False, True])
#cm_display.plot()
#Affichage de l'arbre de décision obtenue après entraînement
plot_tree(model, feature_names=
['preg','plas','pres','skin','insu','mass','pedi','age'],
class_names=["tested_positive","tested_negative"],filled=True)
plt.show()
```

TP

- ▶ Lien vers la base de données des patients diabétiques

<https://drive.google.com/file/d/INfTynWWz3XCAY7ia2XdngfXAYV4MnDd9/view?usp=sharing>

- ▶ Résultats attendus



Apprentissage supervisé: les arbres de décision

Exercice

► Trouver l'arbre de décision correspondant

<i>Emplacement</i>	<i>Type de maison</i>	<i>Revenu</i>	<i>Client antérieur?</i>	<i>Résultat</i>
banlieue	Unifamiliale	élevé	non	Insatisfait
banlieue	Unifamiliale	élevé	oui	Insatisfait
rural	Unifamiliale	élevé	non	Satisfait
ville	Jumelée	élevé	non	Satisfait
ville	Jumelée	bas	non	Satisfait
ville	Jumelée	bas	oui	Insatisfait
rural	Jumelée	bas	oui	Satisfait
banlieue	Rangée	élevé	non	Insatisfait
banlieue	Jumelée	bas	non	Satisfait
ville	Rangée	bas	non	Satisfait
banlieue	Rangée	bas	oui	Satisfait
rural	Rangée	élevé	oui	Satisfait
rural	Unifamiliale	bas	non	Satisfait
ville	Rangée	élevé	oui	Insatisfait

Apprentissage supervisé

Inconvénients

- ▶ L'inconvénient majeur de l'apprentissage supervisé est que les données doivent être étiquetées.
- ▶ Or, dans le monde réel, les données ne le sont généralement pas,
- ▶ Donc l'étiquetage doit se faire à la main, ce qui requiert un temps considérable de la part d'un expert humain.

Apprentissage non-supervisé

- ▶ L'agent reçoit que les données brutes non étiquetées.
- ▶ Aucun expert n'est requis
- ▶ On parle d'observations plutôt que d'exemples
- ▶ Impossible de calculer les taux d'erreurs pour évaluer une potentielle solution
- ▶ L'objectif est de trouver des points communs et des structures cachées dans les données
- ▶ Classer les données en groupes homogènes
 - ▶ Regrouper les données selon leur similarité
 - ▶ C'est ensuite à l'opérateur d'associer ou déduire du sens pour chaque groupe
 - ▶ Le but est de faire ressortir de l'information à partir des données
- ▶ On utilise des algorithmes de classification (clustering) dont les plus connus sont la méthode des **k-means** et la **méthode des sommes pondérées de gaussiennes**.

Apprentissage non-supervisé : Algorithme des K-moyennes (K-means)

- ▶ Partitionner les observations dans K ensembles $S_1, S_2, \dots, S_K$ suivant un ou plusieurs critères, afin de minimiser la distance entre les points à l'intérieur de chaque partition
 - Similaires au sein d'un même groupe
 - Dissimilaires quand ils appartiennent à des groupes différents
- ▶ La similarité est exprimée par le biais d'une mesure de distance
- ▶ Les définitions de distance dépendent de la nature des variables (*intervalles, catégories, booléennes ou ordinales*)
- ▶ Il faut disposer d'une mesure de distance (*distance de Manhattan, distance de Minkowski, distance Euclidienne, etc.*).
- ▶ Il faut choisir K

Apprentissage non-supervisé : Algorithme des K-moyennes (K-means)

- ▶ Les distances expriment une similarité
- ▶ Ex: la *distance de Minkowski* :

$$d(i, j) = \sqrt[q]{(|x_{i1} - x_{j1}|^q + |x_{i2} - x_{j2}|^q + \dots + |x_{ip} - x_{jp}|^q)}$$

où $i = (x_{i1}, x_{i2}, \dots, x_{ip})$ et $j = (x_{j1}, x_{j2}, \dots, x_{jp})$ sont deux objets p -dimensionnels et q un entier positif

- ▶ Si $q = 1$, d est la distance de Manhattan

$$d(i, j) = |x_{i1} - x_{j1}| + |x_{i2} - x_{j2}| + \dots + |x_{ip} - x_{jp}|$$

Apprentissage non-supervisé : Algorithme des K-moyennes (K-means)

- ▶ Si $q = 2$, d est la distance Euclidienne :

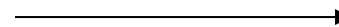
$$d(i, j) = \sqrt{(|x_{i_1} - x_{j_1}|^2 + |x_{i_2} - x_{j_2}|^2 + \dots + |x_{i_p} - x_{j_p}|^2)}$$

- ▶ Propriétés
 - ▶ $d(i, j) \geq 0$
 - ▶ $d(i, i) = 0$
 - ▶ $d(i, j) = d(j, i)$
 - ▶ $d(i, j) \leq d(i, k) + d(k, j)$

Apprentissage non-supervisé : Algorithme des K-moyennes (K-means)

Exemple: distance de Manhattan

	Age	Salaire
Personne1	50	11000
Personne2	70	11100
Personne3	60	11122
Personne4	60	11074



$$d(p1, p2) = 120$$

$$d(p1, p3) = 132$$

Conclusion: p1 ressemble plus à p2 qu'à p3



Apprentissage non-supervisé : Algorithme des K-moyennes (K-means)

- ▶ L'algorithme *k-means* est en 4 étapes :
 1. Choisir k objets formant ainsi k clusters
 2. (Ré)affecter chaque objet O au cluster C_i de centre M_i telle que $\text{dist}(O, M_i)$ est minimale
 3. Recalculer M_i de chaque cluster (le barycentre)
 4. Aller à l'étape 2 si on vient de faire une affectation

Apprentissage non-supervisé : Algorithme des K-moyennes (K-means)

Exemple

- ▶ $A = \{1, 2, 3, 6, 7, 8, 13, 15, 17\}$.
- ▶ Créons 3 clusters à partir de A
- ▶ On prend 3 objets au hasard. Supposons que c'est 1, 2 et 3. Ça donne $C_1 = \{1\}$, $M_1 = 1$, $C_2 = \{2\}$, $M_2 = 2$, $C_3 = \{3\}$ et $M_3 = 3$
- ▶ Chaque objet O est affecté au cluster au milieu duquel, O est le plus proche. 6 est affecté à C_3 car $\text{dist}(M_3, 6) < \text{dist}(M_2, 6)$ et $\text{dist}(M_3, 6) < \text{dist}(M_1, 6)$
On a $C_1 = \{1\}$, $M_1 = 1$,
 $C_2 = \{2\}$, $M_2 = 2$
 $C_3 = \{3, 6, 7, 8, 13, 15, 17\}$, $M_3 = 69 / 7 = 9.86$

Apprentissage non-supervisé : Algorithme des K-moyennes (K-means)

Exemple

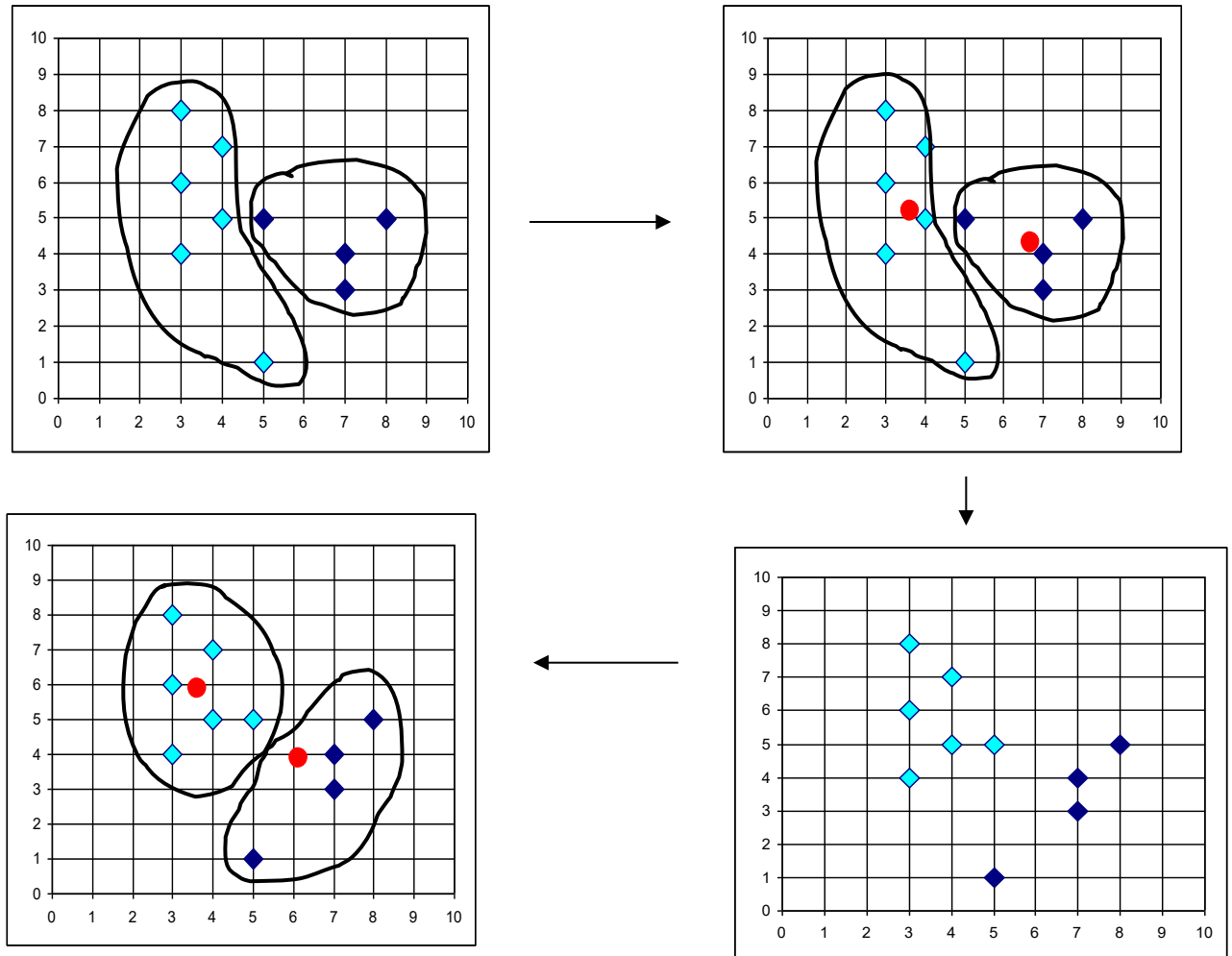
- ▶ $\text{dist}(3, M_2) < \text{dist}(3, M_3) \rightarrow 3$ passe dans C_2 . Tous les autres objets ne bougent pas. $C_1 = \{1\}$, $M_1 = 1$, $C_2 = \{2, 3\}$, $M_2 = 2.5$, $C_3 = \{6, 7, 8, 13, 15, 17\}$ et $M_3 = 66/6 = 11$
- ▶ $\text{dist}(6, M_2) < \text{dist}(6, M_3) \rightarrow 6$ passe dans C_2 . Tous les autres objets ne bougent pas. $C_1 = \{1\}$, $M_1 = 1$, $C_2 = \{2, 3, 6\}$, $M_2 = 11/3 = 3.67$, $C_3 = \{7, 8, 13, 15, 17\}$, $M_3 = 12$
- ▶ $\text{dist}(2, M_1) < \text{dist}(2, M_2) \rightarrow 2$ passe en C_1 . $\text{dist}(7, M_2) < \text{dist}(7, M_3) \rightarrow 7$ passe en C_2 . Les autres ne bougent pas. $C_1 = \{1, 2\}$, $M_1 = 1.5$, $C_2 = \{3, 6, 7\}$, $M_2 = 5.34$, $C_3 = \{8, 13, 15, 17\}$, $M_3 = 13.25$
- ▶ $\text{dist}(3, M_1) < \text{dist}(3, M_2) \rightarrow 3$ passe en 1. $\text{dist}(8, M_2) < \text{dist}(8, M_3) \rightarrow 8$ passe en 2. $C_1 = \{1, 2, 3\}$, $M_1 = 2$, $C_2 = \{6, 7, 8\}$, $M_2 = 7$, $C_3 = \{13, 15, 17\}$, $M_3 = 15$

Plus rien ne bouge

Apprentissage non-supervisé : Algorithme des K-moyennes (K-means)

Exemple

► Illustration



Code Python

```
import pandas as pd
import numpy as np
import sklearn.metrics as sm
import matplotlib.pyplot as plt
from sklearn.cluster import KMeans
from sklearn import datasets
#chargement de base de données iris
iris = datasets.load_iris()
#affichage des données (optionnel)
print(iris.data)
print(iris.feature_names)
print(iris.target)
print(iris.target_names)
#Stocker les données en tant que DataFrame Pandas
x=pd.DataFrame(iris.data)
# définir les noms de colonnes
x.columns=['Sepal_Length','Sepal_width','Petal_Length','Petal_width']
y=pd.DataFrame(iris.target)
```

```
#Cluster K-means
model=KMeans(n_clusters=3)
#adapter le modèle de données
model.fit(x)
#afficher le clustering fait par l'algorithme K-Means
print(model.labels_)
#Visualisez les résultats du classificateur
plt.scatter(x.Petal_Length, x.Petal_width)
#Pour tracer en fonction de la longueur et de la largeur des pétales
colormap=np.array(['yellow','green','blue'])
plt.scatter(x.Petal_Length, x.Petal_width, c=y, s=40)
plt.scatter(x.Petal_Length, x.Petal_width, c=colormap[model.labels_], s=40)
plt.legend(iris.target_names, loc="upper left", title="Ranking")
plt.show()
```

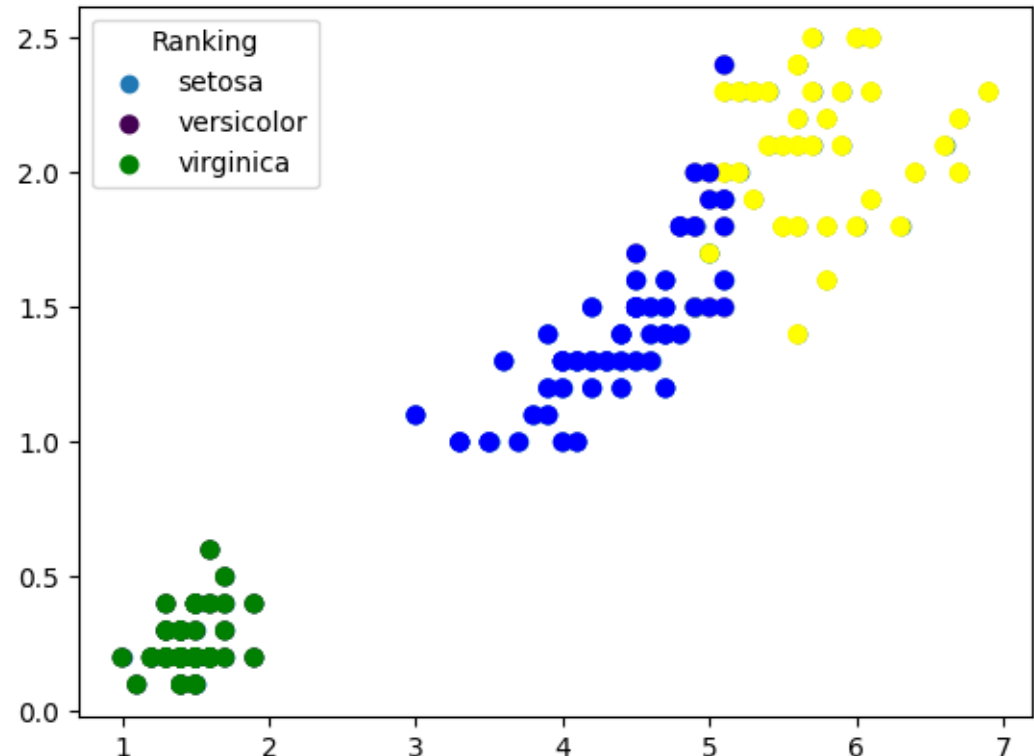
TP

► Lien vers la base de données

from sklearn import datasets

iris = datasets.load_iris()

► Résultats attendus



Apprentissage non-supervisé : Algorithme des K-moyennes (K-means)

Exercice

- Utilisez l'algorithme du k-means et la distance euclidienne pour regrouper les 8 exemples suivants en 3 clusters : $A1(2,10)$, $A2(2,5)$, $A3(8,4)$, $A4(5,8)$, $A5(7,5)$, $A6(6,4)$, $A7(1,2)$, $A8(4,9)$.
1. Donner la matrice de distance basée sur la distance Euclidienne?
On considère comme centre de classes à l'initialisation les points $A1$, $A4$ et $A7$.
 2. Déroulez une itération de l'algorithme de k-means pour ces données et cette initialisation et donnez:
 - a. Les nouveaux clusters
 - b. Les centres de chaque cluster
 - c. Faites une représentation graphique montrant les points d'étude, les clusters et les centres des clusters
 3. Combien d'itérations supplémentaires sont-elles nécessaires pour converger ?
 4. Dessinez les résultats de chaque itération nécessaire.

Apprentissage non-supervisé : Algorithme des K-moyennes (K-means)

▶ Inconvénient

- ▶ N'est pas applicable en présence d'attributs qui ne sont pas de type intervalle (**moyenne=?**)
- ▶ On doit spécifier k (nombre de clusters)
- ▶ Impossible à utiliser quand les clusters sont construits par rapports à des objets inexistants
- ▶ Ne peut pas découvrir les groupes *non-convexes*

Apprentissage non-supervisé : Algorithme des K-moyennes (K-means)

Algorithme (K-means ($K, X, \{x_n\}_{1 \leq n \leq N}$))

Choisir K points $(\mathbf{m}_1, \dots, \mathbf{m}_K)$ de X qui représentent la position moyenne des partitions $S_1, S_2, \dots, S_K$ (par exemple au hasard)

répéter

Assigner chaque observation à la partition la plus proche :

$$S_i = \{ \mathbf{x}_j : \|\mathbf{x}_j - \mathbf{m}_i\| \leq \|\mathbf{x}_j - \mathbf{m}_{i^*}\| \text{ pour tout } i^* = 1, \dots, K \}$$

Mettre à jour la moyenne de chaque partition S_i :

$$\mathbf{m}_i = \frac{1}{|S_i|} \sum_{\mathbf{x}_j \in S_i} \mathbf{x}_j$$

jusqu'à convergence des $\mathbf{m}_i$ (i.e. aucun changement);

Apprentissage par renforcement

- ▶ L'agent doit apprendre à bien agir dans son environnement afin de maximiser sa récompense.
- ▶ À chaque instant, l'agent peut percevoir (complètement ou partiellement) son environnement et choisir ensuite une action à effectuer.
- ▶ Après chaque action, il reçoit une récompense qui peut être positive, négative, ou nulle.
- ▶ À partir de ce feedback et de ses perceptions, l'agent doit apprendre un comportement qui l'amène à obtenir le plus de récompense possible.
- ▶ Parmi l'ensemble des méthodes, nous pouvons citer ceux basées sur les **MDP (Markov Decision Process)**

► RAPPEL SUR LES CHAÎNES DE MARKOV ET LES LOIS DE PROBABILITÉS

Rappel sur les chaînes de Markov (1/18)

Fondé par Andrei Andreevich Markov (1856-1922) un mathématicien russe. Ses travaux sur la théorie des probabilités l'ont amené à mettre au point les chaînes de Markov représentant les prémices de la théorie du calcul stochastique.

1- Les processus stochastiques :

Les *processus stochastiques* sont des modèles mathématiques des phénomènes aléatoires se déroulant et évoluant dans le temps.

Le comportement de tels processus ne peut être décrit à l'avance d'une manière déterministe. Mathématiquement parlant, un processus stochastique est une famille de variables aléatoires $X(t)$ définies sur un même espace de probabilité.

On se donne un ensemble des états E et un ensemble T des instants où l'on considère le système.

T sera:

- soit un intervalle d'origine 0, fini ou infini (cas continu)
- soit l'ensemble $\mathbf{N}$ des entiers naturels (cas discret)

Rappel sur les chaînes de Markov (2/18)

Une chaîne de Markov à temps discret est un processus stochastique $\{X_n, n=0,1,\dots\}$ à temps discret, défini sur un espace d'états S fini ou dénombrable et vérifiant la propriété de Markov

$$P[X_n = i | X_0, \dots, X_{n-1}] = P[X_n = i | X_{n-1}],$$

pour tout i appartenant à S et quelque soit $n \geq 1$.

En mots, l'état courant résume, à lui seul, tout l'historique du système susceptible d'influencer son évolution future.

Propriété:

Une chaîne de Markov à temps discret est homogène (dans le temps) si, pour tout paire d'états (i, j) et tout instant n ,

$$P[X_n = j | X_{n-1} = i] = P[X_{n+k} = j | X_{n+k-1} = i],$$

quel que soit $k \geq 0$.

Rappel sur les chaînes de Markov (3/18)

2- Probabilité de transition et matrice de transition

Soit

$$P_{ij}^{(m)} = P_{ij}(m) = \Pr(X_m = j | X_0 = i)$$

la probabilité de trouver le système à l'état j après m étapes sachant qu'à l'étape 0 il était à l'état i .

$\pi_i^0 = P(X_0 = i)$ la probabilité de trouver le système à l'état i à l'étape 0.

$$\sum_i \pi_i^0 P_{ij}^{(m)} = \sum_i \Pr(X_m = j | X_0 = i) P(X_0 = i)$$

3-La matrice de transition et une matrice stochastique

Matrice stochastique

Une matrice carrée $\mathbf{P} = (p_{ij})$ est stochastique si

- 1) ses éléments sont non négatifs : $p_{ij} \geq 0$ pour tout i et j ;
- 2) la somme des éléments de chacune de ses lignes est égale à 1 : $\sum_j p_{ij} = 1$ pour tout i .

Propriété 2.1 Soit $\mathbf{P}$ une matrice stochastique de taille s finie, alors toute puissance $\mathbf{P}^m$, $m \geq 0$, de $\mathbf{P}$ est une matrice stochastique.

Propriété 2.2 Une matrice de transition d'une chaîne de Markov est une matrice stochastique.



Rappel sur les chaînes de Markov (4/18)

4-Equation de Chapmann-Kolmogorov

Corollaire *Soit $P^{(n)}$, $n \geq 0$, la matrice de transition en n étapes d'une chaîne de Markov. Alors, pour tous entiers non négatifs l et m*

$$P^{(l+m)} = P^{(l)} P^{(m)}$$

ce qui s'écrit sous forme développée

$$p_{ij}^{(l+m)} = \sum_{k \in S} p_{ik}^{(l)} p_{kj}^{(m)} \quad \forall i, j \in S.$$

Rappel sur les chaînes de Markov (5/18)

5-chaînes de Markov homogène

La probabilité conditionnelle d'aller de i à j en m étapes exactement est

$$p_{ij}^{(m)} = P[X_m = j | X_0 = i] = P[X_{n+m} = j | X_n = i] \quad \forall n \geq 1.$$

Cette probabilité est indépendante de n car le processus est homogène et est appelée la probabilité de transition en m étapes de i à j .

La matrice $\mathbf{P}^{(m)}$ dont l'élément (i, j) est égal à $p_{ij}^{(m)}$ est appelée la matrice de transition en m étapes.

Remarque. On a évidemment $p_{ij}^{(1)} = p_{ij}$ et $\mathbf{P}^{(1)} = \mathbf{P}$.

Convention. On adoptera la convention naturelle $\mathbf{P}^{(0)} = \mathbf{I}$, c.-à-d.

$$p_{ij}^{(0)} = \delta_{ij} = \begin{cases} 1 & \text{si } i = j \\ 0 & \text{si } i \neq j \end{cases}$$

Théoreme *Pour tout $m \geq 0$, la probabilité $p_{ij}^{(m)}$ de transition de i à j en m étapes est donnée par l'élément (i, j) de la matrice $\mathbf{P}^m$.*

Sous forme matricielle ce résultat s'écrit : $\mathbf{P}^{(m)} = \mathbf{P}^m$, pour tout $m \geq 0$.

Rappel sur les chaînes de Markov (6/18)

6-Classification des états de la chaîne

La matrice de transition P d'une chaîne de Markov peut être représentée par un graphe orienté G . Les sommets sont les états E de la chaîne. Les arcs représentent les probabilités de transition positive, c'est-à-dire, si un arc relie les sommets associés aux états i et j , alors $p_{ij} > 0$.

Ainsi défini, G est appelé le graphe représentatif, ou graphe de transition, de la chaîne de Markov.

Exemple La matrice de transition d'une chaîne de Markov est :

$$\begin{bmatrix} \frac{1}{4} & \frac{3}{4} & 0 & 0 \\ 0 & \frac{1}{4} & \frac{1}{4} & \frac{1}{2} \\ \frac{1}{2} & 0 & \frac{1}{4} & \frac{1}{4} \\ \frac{1}{2} & 0 & \frac{1}{2} & 0 \end{bmatrix}$$

alors les états de la chaîne de Markov $E=(E_{t_0}, E_{t_1}, E_{t_2}, E_{t_3})$. Le graphe représentatif est sous la forme :

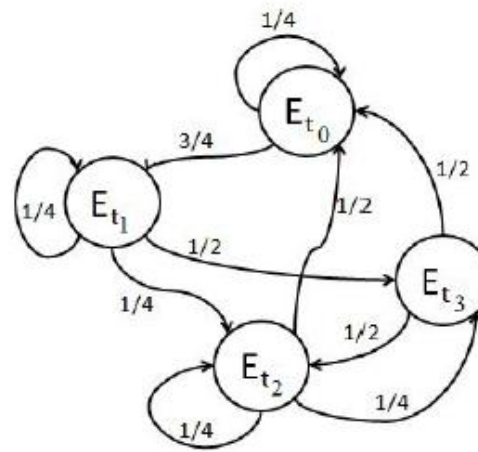


FIGURE – Graphe de transition de la chaîne de Markov.

6-Classification des états de la chaîne

Soit P la matrice de transition d'une chaîne de Markov et G le graphe représentatif de P . L'état j est accessible depuis l'état i s'il existe, dans G , au moins un chemin de i à j . À noter que tout état j est accessible depuis lui-même.

Propriété L'état j est accessible depuis l'état i si et seulement s'il existe $n \geq 0$ tel que $p_{ij}^{(n)} > 0$.

Propriété Les états i et j communiquent s'ils sont accessibles l'un à partir de l'autre, c'est-à-dire, il existe $n \geq 0$ et $m \geq 0$ tels que $p_{ij}^{(n)} > 0$ et $p_{ji}^{(m)} > 0$

La relation i et j communiquent est une relation d'équivalence dont les classes correspondent aux composantes fortement connexes de G . Ainsi, les états i et j communiquent si et seulement s'ils appartiennent à la même classe.

Les classes de la chaîne de Markov correspondent aux composantes fortement connexes de G .

Une classe est persistante si elle correspond à un sommet sans successeur dans G . Si tel n'est pas le cas, la classe est transitoire. Les états d'une classe persistante sont persistants ou récurrents et ceux d'une classe transitoire sont transitoires. Une classe persistante composée d'un seul état est absorbante et un état est absorbant s'il forme, à lui seul, une classe persistante.

Rappel sur les chaînes de Markov (8/18)

6-Classification des états de la chaîne:

Propriété *L'état i est absorbant si et seulement si $p_{ii}=1$ et $p_{ij}=0 \forall i \neq j$.*

Une chaîne de Markov est irréductible si elle ne compte qu'une seule classe. Dans le cas contraire, elle est réductible.

Propriété *Une chaîne de Markov est irréductible si et seulement si son graphe représentatif est fortement connexe.*

Propriété *Une chaîne de Markov est irréductible si et seulement si toutes ses paires d'états communiquent.*

Propriété *Une chaîne de Markov est absorbante si tous ses états persistants sont absorbants.*

NB:

- Si une chaîne est irréductible et si un état est périodique alors la chaîne l'est aussi

7-Périodicité:

La période d de l'état i d'une chaîne de Markov est égale au plus grand diviseur commun de tous les n pour lesquels $p_{ii}^{(n)} > 0$.

L'état i est périodique lorsque $d > 1$ et apériodique lorsque $d = 1$.

Propriété : Si $p_{ii} > 0$, l'état i est apériodique.

Théoreme : Les états d'une classe ont tous la même période.

La période étant une propriété de classe, on parlera de classes périodiques / apériodiques et de chaînes de Markov irréductibles périodiques / apériodiques selon les propriétés de leurs états.

Rappel sur les chaînes de Markov (10/18)

8- Distribution initiale et comportement transitoire:

► Distribution initiale

La distribution des états d'une chaîne de Markov après n transitions est notée $\pi^{(n)}$. Cette distribution est un vecteur de probabilités contenant la loi de la variable aléatoire X_n

$$\pi_i^{(n)} = P[X_n = i], \quad \forall i \in S.$$

La distribution initiale est $\pi^{(0)}$.

Remarque : Si l'état initial est connu avec certitude et est égal à i , on a simplement $\pi_i^{(0)} = 1$ et $\pi_j^{(0)} = 0$ pour tout $j \neq i$.

Rappel sur les chaînes de Markov (11/18)

8- Distribution initiale et comportement transitoire:

► comportement transitoire:

Théoreme : Soit P la matrice de transition d'une chaîne de Markov et $\pi^{(0)}$ la distribution de son état initial. Pour tout $n \geq 1$, on a

$$\pi^{(n)} = \pi^{(n-1)}P \quad \text{et} \quad \pi^{(n)} = \pi^{(0)}P^n.$$

► Distribution invariante

Une distribution π est invariante ou stationnaire si $\pi = \pi P$; donc

$$\lim_{n \rightarrow \infty} \pi^{(n)} \text{ existe}$$

Remarque:

L'étude du comportement à long terme d'une chaîne de Markov cherche à répondre à des questions aussi diverses que

- la distribution $\pi^{(n)}$ converge-t-elle, lorsque $n \rightarrow \infty$?
- si la distribution $\pi^{(n)}$ converge lorsque $n \rightarrow \infty$, quelle est la limite π^* et cette limite est-elle indépendante de la distribution initiale $\pi^{(0)}$?
- si l'état i est persistant, quelle est la proportion du temps passé dans cet état et quel est le nombre moyen de transitions entre deux visites successives de cet état ?
- si l'état i est transitoire, quel est le nombre moyen de visites de cet état ?

Rappel sur les chaînes de Markov (12/18)

Propriété : Une chaîne de Markov possède toujours au moins une distribution invariante.

Théoreme : Une chaîne de Markov possède autant de distributions invariantes linéairement indépendantes que la multiplicité de la valeur propre 1 de sa matrice de transition.

Théoreme : La distribution $\pi^{(n)}$ des états d'une chaîne de Markov converge vers une distribution (invariante) π^* indépendante de la distribution initiale $\pi^{(0)}$ si et seulement si la suite des puissances de la matrice de transition $\mathbf{P}$ de la chaîne converge vers une matrice (stochastique) $\mathbf{P}^*$ dont toutes les lignes sont égales entre elles. De plus, si tel est le cas, chaque ligne de $\mathbf{P}^*$ est égale à π^* .

Rappel sur les chaînes de Markov (13/18)

Théoreme : Soit P la matrice de transition d'une chaîne irréductible et apériodique. Les propriétés suivantes sont vérifiées.

- La matrice P^n tend vers une matrice stochastique P^* lorsque n tend vers l'infini.
- Les lignes de P^* sont toutes égales entre elles.
- $p_{ij}^* > 0$ pour tout $i, j \in S$.
- Pour toute distribution initiale $\pi^{(0)}$,

$$\lim_{n \rightarrow \infty} \pi^{(n)} = \lim_{n \rightarrow \infty} \pi^{(0)} P^n = \pi^*.$$

- π^* est la solution unique du système

$$\begin{cases} \pi P &= \pi \\ \pi 1 &= 1 \end{cases}$$

- π^* est égal à n'importe quelle ligne de la matrice P^* .
- Pour tout $i \in S$, $\pi_i^* = \frac{1}{\mu_i}$
où μ_i est l'espérance du nombre de transitions entre deux visites successives de l'état i .

Rappel sur les chaînes de Markov (14/18)

Remarque. Pour n suffisamment grand, on a $\pi^{(n)} \simeq \pi^*$ et π_i^* est la probabilité que la chaîne se trouve dans l'état i à un instant quelconque. Cette valeur représente aussi la proportion du temps passé dans l'état i .

Exemple

Soit la chaîne de Markov irréductible et apériodique définie par la matrice de transition

$$P = \begin{pmatrix} 1/4 & 0 & 3/4 \\ 1/4 & 1/4 & 1/2 \\ 1/4 & 1/2 & 1/4 \end{pmatrix}.$$

L'unique solution du système

$$\begin{cases} \pi P &= \pi \\ \pi \mathbf{1} &= 1 \end{cases}$$

est $\pi^* = (1/4 \ 3/10 \ 9/20)$.

Le processus passe donc, en moyenne, **30%** du temps dans l'état **2** et il faut, en moyenne, **4** transitions entre deux visites successives de l'état **1**.

9-Les chaînes ergodiques

Une chaîne de Markov est ergodique si elle admet une distribution asymptotique, i.e. si $\lim_{n \rightarrow \infty} \pi^{(n)}$ existe, unique et indépendante de la distribution initiale.

Propriété : *Les chaînes irréductibles et apériodiques sont ergodiques.*

Rappel sur les chaînes de Markov (16/18)

10- Algorithme pour obtenir le vecteur invariant de la chaîne de Markov

Algorithme : Calcul du vecteur invariant.

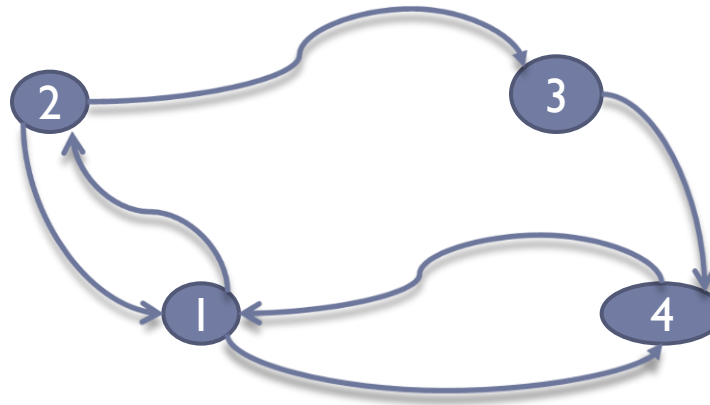
Require: *nb_iteration* // est le nombre d'itérations requises pour que la chaîne de Markov converge, *P* la matrice stochastique de la chaîne de Markov de la tâche.

- 1: $[li, col] = size(P)$ // pour obtenir la taille de *P*.
- 2: $\Pi_0 = rand(1, li)$ // pour générer un vecteur de même dimension que *P* ayant des valeurs aléatoires.
- 3: $sum_li = \sum_{i=0}^{li} \Pi_0[i]$
- 4: **for** $j = 1 : li : j++$ **do**
- 5: $\Pi_0[j] = \Pi_0[j] / sum_li$ // pour assurer que Π_0 est stochastique.
- 6: **end for**
- 7: $X = \Pi_0$
- 8: **for** $i = 1 : nb_iteration : i++$ **do**
- 9: $X = X * P$
- 10: **end for**
- 11: $\Pi = X$
- 12: **return** Π

Rappel sur les chaînes de Markov (17/18)

Exercices:

On considère le graphe suivant:



On définit une chaîne à état $E = \{1, 2, 3, 4\}$. Notons k_i le nombre de flèches partant de i .

Définissons sur E la matrice de transition P par: $P(i, j) = 1/k_i$, si une flèche va de i à j , 0 sinon.

1. Ecrire la matrice de transition P ?
2. Est-elle une chaîne de Markov et est-elle irréductible, apériodique?
3. Calculer $Q = P^2$?
4. La chaîne défini par la matrice de transition Q est-elle irréductible?
5. Trouver l'ensemble des probabilités invariantes pour cette chaîne de Markov de matrice P ?

Rappel sur les chaînes de Markov (18/18)

Exercices:

On étudie le fonctionnement d'une imprimante qui peut être dans 3 états distinct.

Etats 1: attente d'un caractère à imprimer

Etats 2: impression d'un caractère

Etats 3: interruption après avoir reçu un caractère de contrôle.

- Lorsque l'imprimante est en attente elle reçoit un caractère à imprimer avec la probabilité 0,80.

-Lorsqu'elle est en impression elle reçoit

Un caractère normale avec la probabilité 0,95

Un caractère de fin de fichier avec la probabilité 0,04 (l'imprimante retourne dans l'état d'attente)

Un caractère d'interruption avec la probabilité 0,01 (l'imprimante passe alors dans l'état 3).

-Lorsque l'imprimante est dans l'état 3, elle retourne à l'état d'attente avec la probabilité 0,3 sinon elle reste dans l'état 3.

Questions:

1. Montrez que ce système se modélise par une chaîne de Markov à 3 état ?
2. Donnez son graphe et sa matrice de transition ?
3. Périodicité? Irréductibilité? de cette chaîne de Markov
4. Calculer les probabilités stationnaires associées?
5. En régime stationnaire donnez le taux d'utilisation de cette imprimante sachant quelle est ergodique?

Rappel des Lois de distributions usuelles (discrètes):

a) loi de Bernoulli:

- Il convient, de commencer par **considérer que l'événement dont on connaît la probabilité (notée en général p) est un succès**. En conséquence de quoi l'événement contraire constitue un échec. Le succès sera noté $[X_i=1]$ et l'échec sera noté $[X_i=0]$
- On a ainsi défini une variable aléatoire X_i de Bernoulli qui peut prendre 2 valeurs : 0 et 1. On note alors $X_i(\Omega) = \{0;1\}$; $P[X_i=1] = p$ et $P[X_i=0]=1-p=q$

Propriétés de la loi de Bernoulli.

On peut s'assurer que X_i définit bien une variable discrète en vérifiant que $\sum_{k \in X(\Omega)} P[X_i = k] = 1$

- En effet : $\sum_{k \in X(\Omega)} P[X_i = k] = P[X_i=0] + P[X_i=1] = p + 1-p = 1$

- $E(X_i) = \sum_{k \in X(\Omega)} k P[X_i = k] = 0 \cdot P[X_i=0] + 1 \cdot P[X_i=1] = 0 + p = p$

- $E(X_i^2) = \sum_{k \in X(\Omega)} k^2 P[X_i = k] = 0^2 \cdot P[X_i=0] + 1^2 \cdot P[X_i=1] = 0 + p = p$

- $V(X_i) = E(X_i^2) - [E(X_i)]^2 = p - p^2 = p(1-p)$.

- La loi de Bernoulli peut donc se définir à l'aide d'un seul paramètre, p , qui est son espérance mathématique. La variance s'obtient directement à partir de p

$X \rightarrow B(p) \leftrightarrow$

$$\begin{cases} X(\Omega) = \{0;1\} \\ P[X = 1] = p \\ P[X = 0] = q = 1 - p \\ E(X) = p \\ V(X) = pq \end{cases}$$

Rappel des Lois de distributions usuelles (discrètes):

b) Loi binômiale (loi du nombre de succès dans une succession de n épreuves identiques et indépendantes)

- ▶ On considère la succession de n essais identiques et indépendants les uns des autres d'une même épreuve ayant deux issues possibles : le succès avec la probabilité p et l'échec avec la probabilité q=1-p.
- ▶ On définit une variable **X égale au nombre de succès obtenus à l'issue de ces n épreuves.**
- ▶ X correspond à la succession de n épreuves de Bernoulli identiques et indépendantes.
- ▶ On dit alors que X suit une loi Binômiale définie par 2 paramètres : **n** = nombre d'épreuves (de Bernoulli identiques et indépendantes) et **p** = probabilité du succès pour chaque épreuve. On écrit :
- ▶ N= nombre de fois que l'expérience à été recommencée

$$\begin{aligned} \text{▶ } X \rightarrow B(n, p) \leftrightarrow \begin{cases} X(\Omega) = [0; n] \times N \\ P[X = k] = C_n^k p^k q^{n-k} \\ E(X) = np \\ V(X) = npq \end{cases} \end{aligned}$$

Rappel des Lois de distributions usuelles (discrètes):

b) Loi binômiale (loi du nombre de succès dans une succession de n épreuves identiques et indépendantes)

► **Exemple**

On considère une entreprise de service après-vente qui intervient avec retard avec une probabilité égale à 0,25. Un client a appelé à 8 dates différentes.

1°) Préciser la loi de X , son espérance et sa variance

2°) Calculer la probabilité que ce client soit victime d'au moins un retard

3°) Calculer la probabilité que ce client soit victime d'au moins 4 retards sachant qu'il en a subi au moins un.

Rappel des Lois de distributions usuelles (discrètes):

b) Loi binômiale (loi du nombre de succès dans une succession de n épreuves identiques et indépendantes)

► Correction

1°) Appelons succès l'événement dont on connaît la probabilité, à savoir « intervenir avec retard » et notons le $[X_i=1]$; appelons échec l'événement contraire (à savoir « intervenir ponctuellement ») et notons le $[X_i=0]$. On a :

$$P[X_i=1]=0,25 \text{ et } P[X_i=0]=1-0,25=0,75, \text{ avec } X_i(\Omega) = \{0;1\}$$

A l'issue de chaque appel, la probabilité d'intervention avec retard est la même. Les appels peuvent donc être supposés indépendants les uns des autres. Par conséquent, X est la succession de 8 épreuves de Bernoulli identiques et indépendantes. X suit donc une loi binômiale :

$$X \rightarrow B(8, 0,25) \Leftrightarrow \begin{cases} X(\Omega) = [0;8]_{\mathbb{N}} \\ P[X = k] = C_8^k 0,25^k 0,75^{8-k} \\ E(X) = 8 \times 0,25 = 2 \\ V(X) = 8 \times 0,25 \times 0,75 = 1,5 \end{cases}$$

$$2^\circ) P[X \geq 1] = P[(X=1) \text{ ou } (X=2) \text{ ou } \dots \text{ ou } (X=8)] = P[(X=1) \cup (X=2) \cup \dots \cup (X=8)]$$

= $P[X=1] + P[X=2] + \dots + P[X=8]$. Ce calcul peut être effectué en posant successivement $k=1, k=2, \dots, k=8$. Il est toutefois plus simple de remarquer que, dans la mesure où $[X \geq 1]$ est l'événement contraire de $[X < 1]$:

$$P[X \geq 1] = 1 - P[X < 1] = 1 - P[X=0] = 1 - C_8^0 0,25^0 0,75^{8-0} = 1 - 0,75^8 =$$

$$3^\circ) P[X \geq 4 | X \geq 1] = \frac{P[X \geq 4 \cap X \geq 1]}{P[X \geq 1]} = \frac{P[1 \leq X \leq 4]}{P[X \geq 1]}$$

$P[X \geq 1]$ a été calculé à la question précédente. Il reste donc à calculer :

$$P[1 \leq X \leq 4] = P[X=1] + P[X=2] + P[X=3] + P[X=4] \\ = C_8^1 0,25^1 0,75^{8-1} + C_8^2 0,25^2 0,75^{8-2} + C_8^3 0,25^3 0,75^{8-3} + C_8^4 0,25^4 0,75^{8-4}$$

Conclusion : $P[X \geq 4 | X \geq 1] =$

Rappel des Loïs de distributions usuelles (discrètes):

c) Loi hypergéométrique (la loi des sondages):

- ▶ On effectue n tirages **SANS REMISE** dans une population (ou dans une urne) de **taille N** .
- ▶ On recherche alors la loi du nombre de réalisations d'un événement dont **la probabilité** de réalisation, avant que ne commencent les tirages, **est égale à p** .
- ▶ La composition de la population (ou de l'urne) change à l'issue de chaque tirage puisque l'individu (ou la boule) qui vient d'être tiré(e) n'est pas remis(e) dans la population (ou dans l'urne).
- ▶ Soit X le nombre de succès, ou de réalisation de l'événement dont on connaît la probabilité avant que ne commencent les tirages.
- ▶ X définit une loi hypergéométrique de paramètres N , n (échantillon) et p :

$$\left\{ \begin{array}{l} X(\Omega) = [\max(0; n - Nq); \min(n; Np)] \times N \\ P[X = k] = \frac{C_{Np}^k C_{Nq}^{n-k}}{C_N^n} \\ E(X) = np \\ V(X) = npq \frac{N-n}{N-1} \end{array} \right.$$

▶ $X \rightarrow H(N, n, p) \leftrightarrow$

Rappel des Lois de distributions usuelles (discrètes) :

c) Loi hypergéométrique (la loi des sondages):

► Suite de l'exemple précédent

On considère 8 clients différents. On en contacte 4.

On admet qu'un client est mécontent s'il a fait l'objet d'une intervention avec retard.

On note M le nombre de mécontents .

► Donner la loi de M , son espérance et sa variance.

Rappel des Loïs de distributions usuelles (discrètes):

c) Loi hypergéométrique (la loi des sondages):

► Correction

M correspond, dans le cadre d'un sondage, au tirage de 4 clients dans une population de 8 clients sans remise : on peut en effet penser que la personne chargée de l'étude de satisfaction n'interroge chaque client qu'une seule fois.

En outre, avant de commencer les tirages des individus, on sait que la probabilité d'intervention avec retard, donc de mécontentement est de 0,25, donc :

$$X \rightarrow H(8, 4, 0,25) \leftrightarrow \begin{cases} X(\Omega) = [\max(0; 4 - 8 \times 0,75); \min(4; 8 \times 0,25)] \times N \\ P[X = k] = \frac{C_{8 \times 0,25}^k C_{8 \times 0,75}^{4-k}}{C_8^4} \\ E(X) = 4 \times 0,25 \\ V(X) = 4 \times 0,25 \times 0,75 \frac{8-4}{8-1} \end{cases}$$

→

$$\begin{cases} X(\Omega) = [0; 2] \times N \\ P[X = k] = \frac{C_2^k C_6^{4-k}}{C_8^4} \\ E(X) = 1 \\ V(X) = 0,75 \frac{4}{7} \end{cases}$$

Rappel des Lois de distributions usuelles (discrètes):

d) Loi géométrique (Loi du nombre du premier succès; du nombre de paquets transmis avec succès; etc.)

- ▶ On effectue n épreuves (de Bernoulli) identiques et indépendantes dont la probabilité de succès est p pour chacune d'elles. La probabilité de l'échec pour chacune d'elles est notée $q=1-p$.
- ▶ On appelle **X le rang du 1er succès**, donc:
- ▶ $P[X=k]$ signifie : probabilité que le 1er succès soit obtenu à l'issue de la k -ième épreuve.
- ▶ En d'autres termes, les $k-1$ premières épreuves se sont soldées par des échecs et la k -ième par un succès.
- ▶ Cette distribution est la seule discrète avec la propriété sans mémoire

Géométrique de base

$$X \rightarrow G(p) \Leftrightarrow \begin{cases} \{X(\Omega) = [1; +\infty[\\ P[X = k] = q^{k-1} p \\ E(X) = 1 / p \\ V(X) = q / p^2 \end{cases}$$

Géométrique modifiée(cas

$$\begin{cases} \{X(\Omega) = [1; +\infty[\\ P[X = k] = p^k q \\ E(X) = p / q \\ V(X) = p / q^2 \end{cases} \text{truquée)}$$

Aussi appelé la loi de Pascal qui est la généralisation de la loi géométrique de base lorsque l'on s'intéresse à l'obtention pour la $k^{\text{ème}}$ fois de l'événement de probabilité p

Rappel des Loïs de distributions usuelles (discrètes):

d) Loi géométrique (Loi du nombre du premier succès)

Suite de l'exemple précédent

Soit Y la loi du rang du 1er retard. Préciser la loi de Y , son espérance et sa variance

Les appels qui définissent une loi de Bernoulli étant identiques et indépendants, le temps d'attente du 1er retard définit une loi géométrique de même paramètre que la loi de Bernoulli précédemment évoquée :

$$Y \rightarrow G(0,25) \Leftrightarrow \begin{cases} \{X(\Omega) = [1; +\infty[\\ P[X = k] = 0,75^{k-1} 0,25 \\ E(X) = 1 / 0,25 = 4 \\ V(X) = 0,75 / 0,25^2 \end{cases}$$

Rappel des Lois de distributions usuelles (discrètes):

e) Loi de poisson (Loi du temps des inter arrivées)

La loi de Poisson résulte de la convergence de la loi binômiale. En d'autres termes, lorsque n est très grand ($n \geq 30$) et sous réserve que 2 autres conditions soient également satisfaites ($p \leq 0,1$ et $np < 15$), on peut remplacer la loi binômiale $B(n ; p)$ par une loi de poisson de paramètre λ avec $\lambda = np$. On a alors :

$$X \rightarrow P(\lambda) \Leftrightarrow \begin{cases} X(\Omega) = N \\ P[X = k] = e^{-\lambda} \cdot \frac{\lambda^k}{k!} \\ E(X) = V(X) = \lambda \end{cases}$$

Exemple:

Nbre de requêtes au serveur dans un intervalle de temps t

Nbre de panne par unité de temps

Etc.

Rappel des Loïs de distributions usuelles (discrètes):

e) Loi de poisson (Loi du temps entre deux arrivées)

Exemple

X suit une loi de Poisson de paramètre 4.

1°) Calculer $P[X=5]$

2°) Préciser $E(X)$ et $V(X)$

3°) On démontre que si X et Y sont deux variables indépendantes avec $X \rightarrow P(\mu)$ et $Y \rightarrow P(\lambda)$ alors $X+Y \rightarrow P(\lambda + \mu)$ En déduire la variance de X+Y

Utiliser la formule du binôme de Newton ci-dessous pour démontrer $X+Y \rightarrow P(\lambda + \mu)$

$$\sum_{i=0}^n C_k^i a^i b^{k-i} = (a + b)^n$$

Rappel des Lois de distributions usuelles (discrètes):

e) Loi de poisson (Loi du temps entre deux arrivées)

Correction:

$$1^{\circ}) P[X=5] = e^{-4} \cdot \frac{4^5}{5!}$$

2°) $E(X)=V(X)=4$ (la moyenne et la variance sont égales au paramètre de la loi de Poisson)

3°) Pour la raison qui vient d'être évoquée : $V(X+Y)$ = paramètre de la loi de $X+Y$ soit

$$\lambda + \mu$$

Rappel des Loïs de distributions usuelles (continues):

f) Loi exponentielle (Loi des temps des services)

- ▶ Cette distribution est la seule continue avec la propriété sans mémoire
- ▶ Durée entre deux arrivées successifs de client dans un réseau
- ▶ Temps de services d'un serveur dans un réseau
- ▶ Durée de vie d'une composante ou pour réparer un composant défectueux
- ▶ Etc.

Caractéristique

La fonction de distribution de probabilité (PDF) est donnée par:

$$F(x) = 1 - e^{-\lambda x} \quad 0 \leq x \leq \infty$$

et la fonction de densité de probabilité par:

$$f(x) = \lambda e^{-\lambda x} \quad x > 0$$

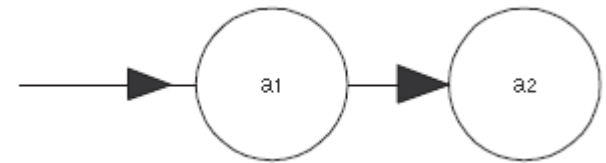
Rappel des Lois de distributions usuelles (continues):

g) Loi hypo-exponentielle (Loi des temps des services séquentiels et des phases séquentiels)

- ▶ Si le processus consiste à des phases **séquentiels**
- ▶ Les temps passés dans chaque phase sont indépendants et de **distribution exponentielle**
- ▶ Alors le temps total passé dans toutes les phases est de **distributions hypo-exponentielle de paramètres r_i un pour chaque phase.**

Example:

- ▶ Traitement séquentiel de requête de base de données,
- ▶ Traitement séquentiel de tâches



Caractéristique

Une variable aléatoire X à deux phases de distribution hypo-exponentielle a deux paramètres λ_1 et λ_2 . Ses fdp et FDP sont données par :

$$f(t) = \frac{\lambda_1 \cdot \lambda_2}{\lambda_2 - \lambda_1} (e^{-\lambda_1 t} - e^{-\lambda_2 t}) \quad t > 0$$

$$F(t) = 1 - \frac{\lambda_2}{\lambda_2 - \lambda_1} e^{-\lambda_1 t} + \frac{\lambda_1}{\lambda_2 - \lambda_1} e^{-\lambda_2 t} \quad t \geq 0$$

Rappel Mathématiques

Rappel des Loïs de distributions usuelles (continues):

h) Loi hyper-exponentielle (Loi des temps des services parallèles et des phases parallèles)

- ▶ Si le processus consiste à des phases **parallèles**.
- ▶ Les temps passés dans chaque phase sont indépendants et de **distribution exponentielle**
- ▶ Alors le temps total passé dans toutes les phases est de **distributions hyper-exponentielle de paramètres r , un pour chaque phase**.

Exemple:

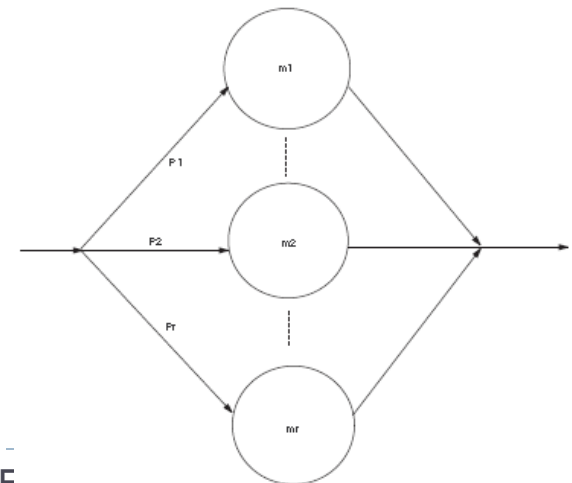
- ▶ Traitement parallèle de requête de base de données,
- ▶ Traitement parallèle de tâches

Caractéristique

Les FDP et fdp d'une variable aléatoire à k phases est données par:

$$f(t) = \sum_{i=1}^k \alpha_i \lambda_i e^{-\lambda_i t} \quad \text{avec } t, \lambda_i, \alpha_i > 0, \text{ et } \sum_{i=1}^k \alpha_i = 1$$

$$F(t) = \sum_{i=1}^k \alpha_i (1 - e^{-\lambda_i t}), \quad t > 0$$



Rappel des Lois de distributions usuelles (continues):

i) Loi de Weibull (Mesures de fiabilité et de temps de bon fonctionnement)

- ▶ Très utilisée dans plusieurs domaines (électronique, mécanique, ...).
- ▶ Elle permet de modéliser en particulier de nombreuses situations d'usure de matériel.
- ▶ Elle caractérise le comportement du système dans les trois phases de vie : période de jeunesse, période de vie utile et période d'usure ou vieillissement.
- ▶ Dans sa forme la plus générale, la distribution de Weibull dépend des trois paramètres suivants : β , η et γ
- ▶ La densité de probabilité d'une loi de Weibull a pour expression

$$f(t) = \frac{\beta}{\eta} \left(\frac{t-\gamma}{\eta} \right)^{\beta-1} e^{-\left(\frac{t-\gamma}{\eta} \right)^{\beta}} \quad t \geq \gamma$$

β est le paramètre de forme ($\beta > 0$)
 η est le paramètre d'échelle ($\eta > 0$)

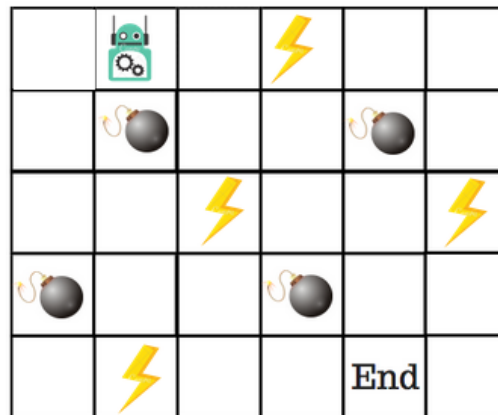
- ▶ La fonction de fiabilité s'écrit: $R(t) = e^{-\left(\frac{t-\gamma}{\eta} \right)^{\beta}}$ γ est le paramètre de position ($\gamma \geq 0$)

- ▶ Le taux de défaillance est donnée par: $\lambda(t) = \frac{\beta}{\eta} \left(\frac{t-\gamma}{\eta} \right)^{\beta-1}$

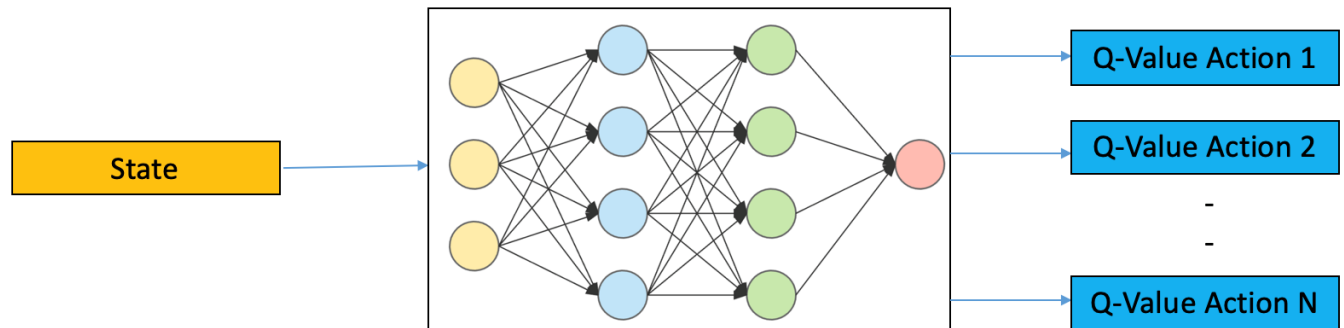
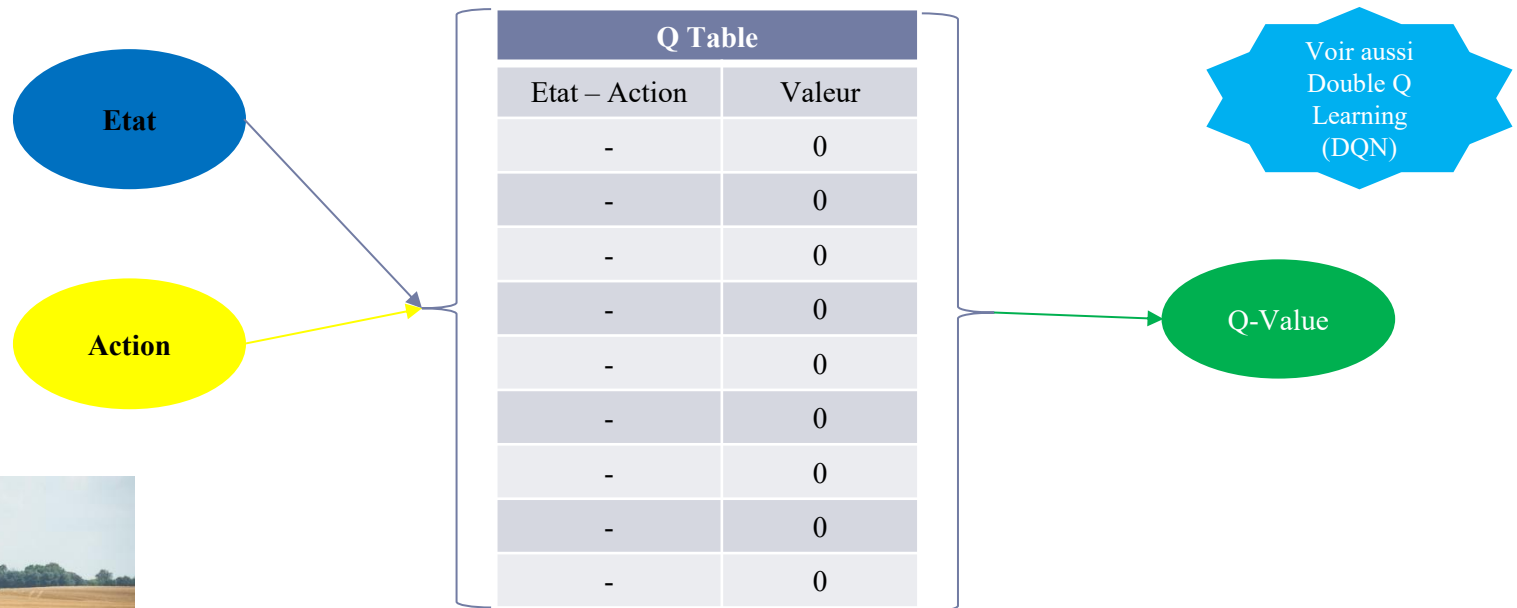
Le Q learning

Q learning

- ▶ C'est un autre algorithme de base de l'apprentissage par renforcement (*Reinforcement Learning (RL)*).
 - ▶ cherche à trouver la meilleure action à entreprendre compte tenu de l'état actuel.
 - ▶ est capable d'effectuer des actions aléatoires sans tenir compte de la **politique** à l'état actuel.
 - ▶ cherche surtout à apprendre une **politique** qui maximise la récompense totale.
 - ▶ commence par effectuer un choix entre l'exploration et l'exploitation des données issu de la perception de l'environnement.
- ▶ L'agent peut opter pour l'exploration s'il n'a aucune expérience ou l'exploitation des données.
 - ▶ Une fois qu'il a effectué une action, l'agent passe à l'étape suivante où il prend de nouvelles décisions.
 - ▶ Au cours de son apprentissage, l'agent IA progresse lentement en fonction de l'expérience acquise.



Q Learning



Deep Q Learning

Q Learning

- Il se base principalement sur la fonction État-Action notée Q

$$Q : S \times A \rightarrow \mathbb{R}$$

$$\text{New } Q(s, a) = Q(s, a) + \alpha [R(s, a) + \gamma \max_{a'} Q(s', a') - Q(s, a)]$$

(Facteur d'apprentissage) Learning Rate α
 (Facteur d'actualisation [0,1] dépend du délai entre les 2 étapes) Discount Rate γ
 New Q value for the state and action
 Current Q values (Valeur actuelle de Q)
 Reward for taking an action in a state (Récompense reçue en faisant l'action a à l'état s)
 Maximum expected future reward (Meilleure action a' possible en s')
 Current Q values (Valeur actuelle de Q)

(Nouvelle valeur de Q à l'état s après l'action a)
 (Récompense reçue en faisant l'action a à l'état s)

Initialized

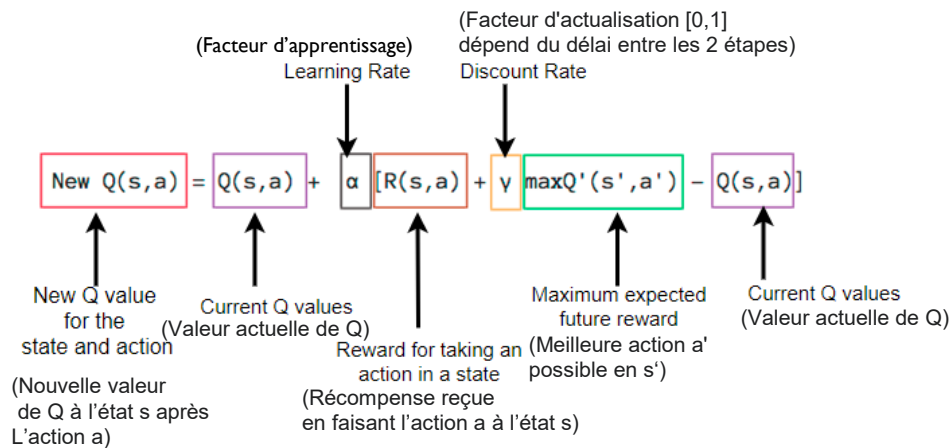
Q-Table		Actions					
		South (0)	North (1)	East (2)	West (3)	Pickup (4)	Dropoff (5)
States	0	0	0	0	0	0	0
	.	.	.	.	.	.	.
	.	.	.	.	.	.	.
	.	.	.	.	.	.	.
	327	0	0	0	0	0	0
	.	.	.	.	.	.	.
	.	.	.	.	.	.	.
	499	0	0	0	0	0	0

Training

Q-Table		Actions					
		South (0)	North (1)	East (2)	West (3)	Pickup (4)	Dropoff (5)
States	0	0	0	0	0	0	0
	.	.	.	.	.	.	.
	.	.	.	.	.	.	.
	.	.	.	.	.	.	.
	328	-2.30108105	-1.97092096	-2.30357004	-2.20591839	-10.3607344	-8.5583017
	.	.	.	.	.	.	.
	.	.	.	.	.	.	.
	499	9.96984239	4.02706992	12.96022777	29	3.32877873	3.38230603

Q Learning

- Il se base principalement sur la fonction État-Action notée Q
- $$Q : S \times A \rightarrow \mathbb{R}$$



- Si $\alpha=0$ alors l'agent n'a rien appris
- Si $\alpha=1$ alors l'agent ignore toujours tout ce qu'il a appris jusqu'à présent et ne considérera que la dernière information.
- En milieu déterministe $\alpha=1$ mais en milieu stochastique c'est souvent à 0,1
- Si $\gamma=0$ l'agent considère seulement les récompenses courantes
- Si $\gamma \sim 1$ l'agent fait intervenir les récompenses plus lointaines

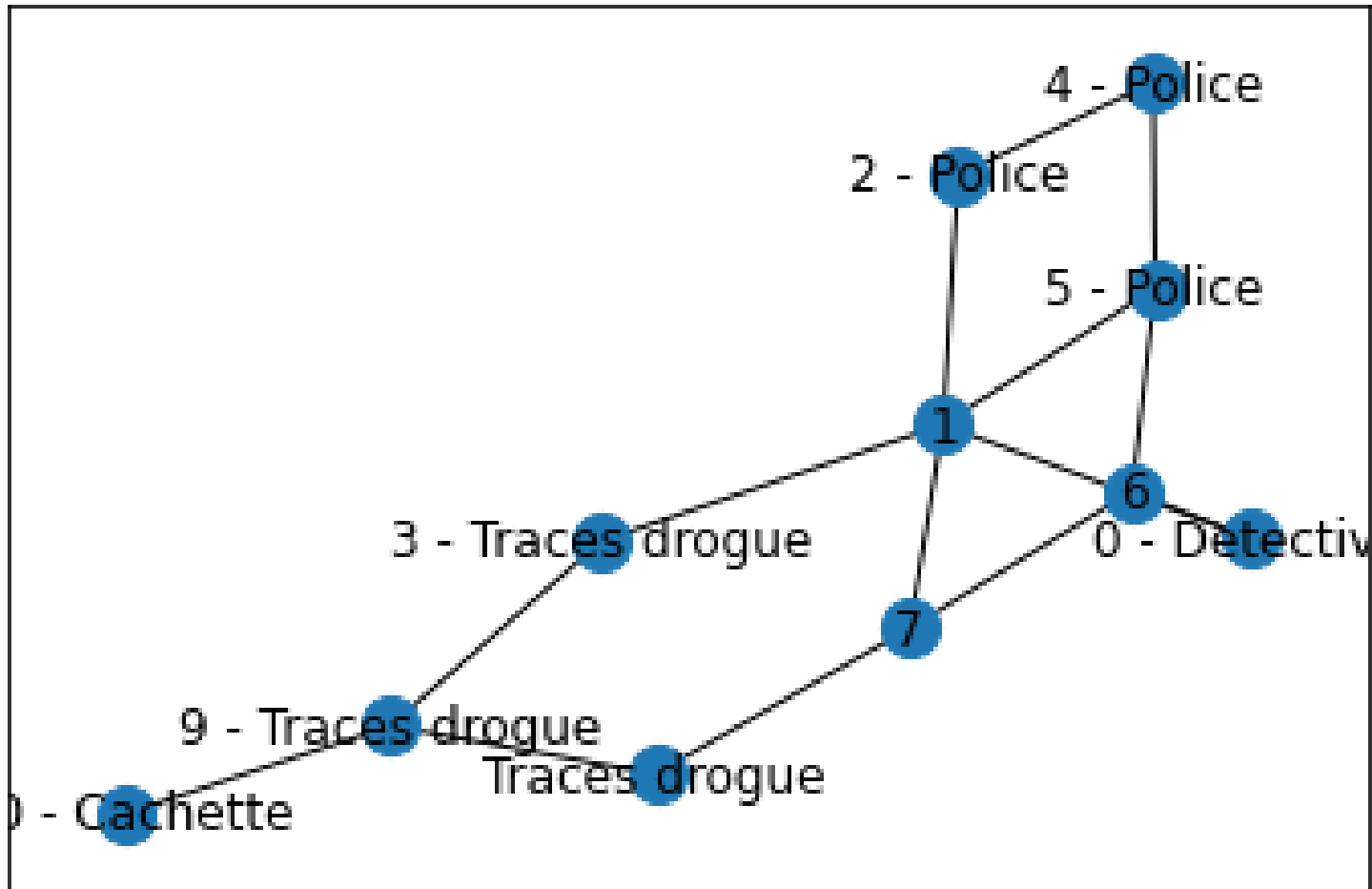
Algorithme du Q Learning

► Pseudo-code du Q-Learning

- Initialiser $Q[s, a]$ pour tout état s , toute action a de façon arbitraire, mais $Q(\text{état terminal}, a) = 0$ pour toute action a
- répéter
 - . initialiser l'état s arbitrairement
 - répéter
 - . choisir une action a depuis s en utilisant la politique spécifiée par Q (par exemple ϵ -greedy)
 - . exécuter l'action a
 - . observer la récompense r et l'état s'
 - . $Q[s, a] = Q[s, a] + \alpha[r + \gamma \max_{a'} Q(s', a') - Q(s, a)]$
 - . $s = s'$
 - . $a = a'$
 - jusqu'à ce que s soit l'état terminal

Q Learning

► Exemple



Apprentissage par renforcement

Recherche pour les étudiants: résumé de 15 diapos (Définitions formelles et un exemple déroulé)

- ▶ MDP
- ▶ DEC-MDP
- ▶ POMDP
- ▶ DEC-POMDP
- ▶ RÉSEAUX D'AUTOMATES STOCHASTIQUES
- ▶ DOUBLE Q-LEARNING
- ▶ DQN (DEEP Q-NETWORKS)
- ▶ DOUBLE DQN
- ▶ ...

Bibliographie

- <http://ocw.mit.edu/courses/electrical-engineering-and-computer-science/6-825-techniques-in-artificial-intelligence-sma-5504-fall-2002/lecture-notes/>
- <https://zestedesavoir.com/tutoriels/1760/un-peu-de-machine-learning-avec-les-svm/>
- <https://www.geeksforgeeks.org/ml-reinforcement-learning-algorithm-python-implementation-using-q-learning/?ref=rp>
- <https://hermit-notebook.site/fr/notebook/computer-sciences/artificial-intelligence/machine-learning/taxonomy-of-machine-learning/>
- <https://www.geeksforgeeks.org/epsilon-greedy-algorithm-in-reinforcement-learning/>
- <https://openclassrooms.com/fr/courses/4011851-initiez-vous-au-machine-learning/4020611-identifiez-les-differents-types-dapprentissage-automatiques>
- <http://aima.cs.berkeley.edu/>
- <http://www.afia-france.org/>
- <http://ria.revuesonline.com/>
- <http://www.aaai.org/>
- <http://www.aaai.org/Magazine>
- <http://www.sigart.org/>
- <http://ai.stanford.edu/~nilsson/>
- <http://www-formal.stanford.edu/jmc/>
- <http://web.media.mit.edu/~minsky/>
- <http://www.jair.org/>
- <http://www.ijcai.org/>
- <http://www.ida.liu.se/ext/aijd/>
- <http://www.eccai.org/>
- <http://zhar.net/howto/>
- <http://www.etaij.org/>
- <http://www.ida.liu.se/~ulfni53/lpp/>

etc.